

ECOM EXPERTS

Orderfees shipping

Manual de usuario, tecnico y de desarrollo

orderfees_shipping · 08/09/2026

Contenido

1. Para qué sirve	
2. Dónde está	
 Requisito imprescindible	
3. El listado de reglas	
4. Crear una regla: las 3 pestañas	
Pestaña 1 — Information	
Pestaña 2 — Conditions (¿cuándo se aplica?)	
Pestaña 3 — Actions (¿qué hace?)	
5. Fórmulas	
Reglas de escritura	
Variables que puedes usar	
Funciones que puedes usar	
Ejemplos que funcionan	
Errores típicos	
6. La prioridad: lo que más equivocaciones causa	
7. Recetas listas para copiar	
8. Cosas que conviene saber (para no llevarse sorpresas)	
9. Algo no cuadra: ¿por qué este pedido tiene ese coste de envío?	
Usa el Simulador	
Pistas rápidas, sin abrir nada	
Para pedidos muy antiguos	
1. Qué hace	
2. Arquitectura	
2.1 Ficheros clave	
2.2 Flujo de ejecución completo	
2.3 Hooks	
3. Modelo de datos	
3.1 Tabla maestra PREFIX_of_shipping_rule (install.sql:2-46)	
3.2 Satélites de restricción simples (PK compuesta, M:N)	
3.3 Árbol de productos (3 niveles)	
3.4 Árboles de 2 niveles (valores en CSV en la columna value)	
3.5 Relaciones (todas lógicas, sin FK)	
3.6 Historial de esquema (upgrades relevantes)	
4. Motor de reglas	
4.1 Bitmask type (ShippingRule.php:112-131)	
4.2 Condiciones — ShippingRule::check() (:680-1186)	
4.3 Motor de dimensiones (:895-1008)	
4.4 Motor de "package" (tarifa por tramos de peso volumétrico) — orderfees_shipping.php:236-305	
4.5 Motor de condiciones de producto — restrictionsProducts() (:1565-1766)	
4.6 Cálculo del valor — ShippingRule::value(\$params) (:1188-1383)	

5. Fórmulas (parser matemático)

6. Configuration keys

7. Trazabilidad: qué queda registrado

7.1 El Simulador (añadido por Ecom Experts)

8. Requisitos y compatibilidad

1. Mapa de intervención rápida

2. Puntos de extensión SIN tocar el módulo

3. Cómo averiguar por qué un pedido recibió un coste de envío determinado

3.1 El problema de raíz

3.2 Qué NO es reconstruible a posteriori (ni instrumentando ahora)

3.3 Queries SQL de diagnóstico

3.4 Procedimiento manual, paso a paso

4. Plan de instrumentación (para responder la pregunta EN AUTOMÁTICO)

Fase 0 — Flag de depuración

Fase 1 — Logger

Fase 2 — Motivo de rechazo en check() — EL PUNTO CRÍTICO

Fase 3 — Trazar la resolución del bucle

Fase 4 — Persistencia por pedido (resuelve el problema de raíz)

Alternativa sin tocar el módulo (menos potente, cero riesgo)

5. Bugs confirmados

6. Riesgos de arquitectura y operación

7. Checklist de verificación en la tienda del cliente

8. Notas de mantenimiento

1. Dónde está

2. Los 3 modos de entrada

a) Un carrito que monto a mano

b) La dirección de un cliente real

c) Un pedido existente ← el caso "cobró de más / de menos"

3. Qué te devuelve

La traza, columna a columna

4. Cómo sabe "por qué NO aplicó" — sondas de aislamiento

Autovalidación

5. Seguridad: el carrito temporal

6. Configuración

7. Ficheros

8. Limitaciones conocidas

Guía para el comerciante. No requiere conocimientos técnicos. Módulo: **orderfees_shipping** ("Advanced Shipping cost", de motionSeed).

1. Para qué sirve

PrestaShop, de serie, solo sabe cobrar los portes **por peso** o **por precio**, y por **zona**. Este módulo añade encima un sistema de **reglas** que te permite decidir el coste de envío con muchísimo más detalle.

Una regla es una frase del tipo:

SI el pedido cumple *estas condiciones* → **ENTONCES** haz esto con los gastos de envío.

Por ejemplo:

- Si el pedido supera 50 € → envío gratis.
- Si el código postal empieza por 07 (Balears) → suma 15 € al porte.
- Si el producto mide más de 3 metros de contorno → suma 25 €.
- Si son más de las 14:00 → oculta el transportista urgente.
- Si el bulto pesa entre 5 y 10 kg volumétricos → cobra 9,00 € (y olvídate del precio de PrestaShop).

Puedes tener tantas reglas como quieras, y se aplican en el orden que tú decidas.

2. Dónde está

Back-office → Transporte → "Advanced Shipping cost"

El botón "Configurar" del módulo no abre ninguna pantalla propia: te lleva directamente a este listado. **Toda la configuración del módulo son las reglas.**

⚠ Requisito imprescindible

El módulo **no funciona si los overrides están desactivados**.

Ve a **Parámetros avanzados** → **Rendimiento** → "**Desactivar todos los overrides**" y asegúrate de que está en **NO**. Si está en **SÍ**, el módulo te mostrará un aviso permanente y **ninguna regla se aplicará**.

3. El listado de reglas

Columna	Qué es
ID	Identificador interno.
Name	El nombre que le pusiste. Es el único texto libre: ponlo descriptivo ("Recargo Balears", "Envío gratis > 50 €").
Priority	El orden en que se procesan. La regla con prioridad 1 se evalúa antes que la 2 . Esto importa mucho (ver §6).
Status	Semáforo verde/rojo. Se puede pulsar directamente para activar/desactivar sin entrar a editar.

Acciones por fila: Editar · **Duplicar** · Borrar.

Duplicar crea una copia completa e independiente, con todas sus condiciones. Es la forma más rápida de hacer variantes ("la misma regla, pero para Canarias").

4. Crear una regla: las 3 pestañas

Al pulsar "**Add new rule**" verás un formulario con 3 pestañas. **Hay un solo botón Guardar** que envía las tres a la vez.

Pestaña 1 — Information

Campo	Qué poner
Name	Nombre de la regla. Solo lo ves tú.
Priority	Número de orden. Por defecto 1. Ver §6.
Active	Sí/No. Forma segura de "apagar" una regla sin borrarla.

Pestaña 2 — Conditions (¿cuándo se aplica?)

Concepto clave: cada bloque tiene una casilla. **Casilla desmarcada = no restringe nada** (la regla vale para todos). **Todas las condiciones que marques se combinan con Y:** deben cumplirse **todas a la vez**.

a) Cliente

Campo	Qué hace
Customer	Autocompletado (busca por ID, email, nombre o apellido). Limita la regla a un solo cliente . Déjalo en blanco para todos.

b) Importes

Campo	Qué hace
Importe mínimo	La regla se aplica si el importe es mayor o igual que este valor.
Moneda	En qué moneda expresas el mínimo (se convierte sola a la del cliente).
Impuestos	<i>Excluidos</i> (sin IVA) o <i>Incluidos</i> (con IVA).
Sobre qué se mide	Total del pedido (sin portes) · Solo los productos de la condición de productos · El coste de envío .
Importe máximo	La regla se aplica si el importe es menor (estrictamente) que este valor.

Combinando mínimo y máximo defines una **horquilla**: "entre 30 € y 100 €".

c) Restricciones de doble lista

Todas funcionan igual: una lista a la izquierda ("no seleccionados") y otra a la derecha ("seleccionados"), con botones para mover elementos.

Bloque	Qué restringe
Countries	Países (según la dirección de entrega).
Zones	Zonas (Europa, Norteamérica...).
States	Provincias / estados.
Cities	Ciudades — solo disponible si tienes un catálogo de ciudades instalado.
Carriers	Transportistas.
Customer groups	Grupos de clientes (Invitado, Cliente, Mayorista...).
Genders	Tratamientos (Sr. / Sra.).
Shops	Tienda (solo en multitienda).
Compatibilidad con otras reglas	Izquierda = reglas NO combinables · Derecha = reglas combinables. Si no marcas la casilla, esta regla es compatible con todas.

⚠ Ojo con un comportamiento del formulario: si mueves **todos** los elementos a la derecha (dejando la lista izquierda vacía), al guardar el módulo **desmarca automáticamente la casilla** — interpreta que "seleccionarlo todo" es lo mismo que "no restringir". El efecto final es correcto, pero al volver a abrir la regla verás la casilla desmarcada.

⚠ **Compatibilidad entre reglas es bidireccional.** Al marcarla, el módulo **modifica automáticamente las otras reglas** para mantener la matriz coherente. Si un día ves que se te han activado casillas solas en otras reglas, es por esto.

d) Cities (por texto)

Cuando no tienes catálogo de ciudades, defines las ciudades escribiéndolas.

Campo	Qué hace
Grupo	Los grupos se combinan con Y ; las líneas dentro de un grupo, con O .
País	El país al que aplica esa línea.
Ciudades	Nombres separados por coma: <code>Madrid, Getafe, Leganés</code> .

e) Zip codes (códigos postales)

Campo	Qué hace
Grupo	Y entre grupos, O dentro del grupo.
País	El país al que aplica esa línea.
Operador	<code>begin</code> (empieza por) ← el más útil · <code>end</code> (acaba en) · <code>=</code> · <code>></code> · <code><</code> · <code>>=</code> · <code><=</code> · <code>!=</code> .
Valor	El/los CP, separados por coma.

Ejemplo Baleares: país = España, operador = **empieza por**, valor = `07` .

f) Products (productos, categorías, atributos...)

Campo	Qué hace
Cantidad del grupo	"El carrito debe contener al menos N producto(s) que cumplan las reglas siguientes."
Tipo	Qué se filtra: Productos · Atributos · Características · Categorías · Fabricantes · Proveedores · Categoría por defecto · Sin stock .
Selección	Se abre una ventana emergente con doble lista para elegir los elementos concretos. Al cerrarla, el campo muestra un resumen.

g) Dimensions (dimensiones)

Campo	Qué hace
Base de cálculo	Por producto (cada artículo por separado) · Por producto con cantidad · Conjunto de todos los productos (suma del carrito).
Tipo	Ancho · Alto · Profundidad · Peso · Volumen · Combinada (L+A+Al) · Combinada con contorno (L+2A+2Al ← la fórmula que usan UPS/FedEx/DHL).
Operador	<code>=</code> · <code>></code> · <code><</code> · <code>>=</code> · <code><=</code> · <code>!=</code> .
Valor	El número. La unidad se muestra al lado.

⚠ Si **ningún producto del carrito tiene dimensiones rellenadas**, esta condición **se ignora** y la regla se aplica igualmente. Rellena las dimensiones de tus productos.

h) Package dimensions (peso volumétrico / bultos)

El bloque más potente y el más complejo. Sirve para replicar **la tarifa de tu agencia de transporte**.

A nivel de grupo:

Campo	Qué hace
Unidad	kg/m ³ o cm ³ /kg .
Ratio predefinido	Los ratios estándar de las agencias: 333 / 250 / 200 / 166,667 / 150 / 142,857 kg/m³ · 5000 / 6000 / 7000 cm³/kg.
Ratio manual	Alternativa si tu agencia usa otro. Si eliges un predefinido y dejas este vacío, se rellena solo. No hay que poner los dos.

A nivel de línea (cada tramo de peso):

Campo	Qué hace
Range start / Range end	El tramo de peso (de X kg a Y kg).
Round	Redondeo: a qué múltiplo se redondea el peso facturable. 1 = al kilo entero hacia arriba.
Divider	Divisor: permite tarificar "por cada N kg". Si vale 1 , el importe es fijo para todo el tramo.
Value	El importe que cobras en ese tramo.
Tax	Si se le aplica el IVA del transportista.

Cómo calcula el módulo el peso facturable:

1. Suma el peso real de todos los productos (peso × cantidad).
2. Calcula el **peso volumétrico** a partir del volumen y del ratio.
3. **Se queda con el MAYOR de los dos.** (Es exactamente lo que hacen las agencias.)
4. Lo redondea según **Round** y busca en qué **tramo** cae.
5. Cobra el **Value** de ese tramo.

 **Usa un solo grupo de "Package dimensions" por regla.** Si pones varios, solo se aplica el último.

Pestaña 3 — Actions (¿qué hace?)

Tipo de acción

Tipo	Qué hace
Free shipping	Envío gratis. Pone el porte a 0 y corta: ninguna regla posterior se evalúa.
Percent	Un porcentaje . Ver "Aplicar a" abajo.
Amount	Un importe fijo (con su moneda y su grupo de IVA).
Formula	Una fórmula matemática . Ver \$5.
Show only selected carriers	Ocultar los transportistas que no estén en la lista de la pestaña Conditions. Útil para "el urgente solo hasta las 14:00".
Weight	No cambia el precio directamente: cambia el peso que PrestaShop usa para elegir la tarifa. Es una forma indirecta de ajustar los portes nativos (p. ej. sumar el peso del embalaje).

Aplicar a (solo para Percent)

Opción	Qué hace
Total del pedido	El % se calcula sobre el total del pedido (sin portes).
Un producto concreto	El % se calcula sobre ese producto.
Los productos seleccionados	El % se calcula sobre los productos que cumplan tus condiciones de producto.

Aplicar si

Opción	Qué hace
Al menos un producto cumple	Basta con que uno de los productos del carrito cumpla las condiciones. Es lo normal.
Todos los productos cumplen	Todos los productos del carrito deben cumplirlas.

Compatibilidad con los gastos de envío básicos — EL CAMPO MÁS IMPORTANTE

Opción	Qué hace
Sumar al coste básico	El resultado de la regla se suma al porte que calculó PrestaShop. El bucle continúa con las siguientes reglas. Es el modo normal para recargos.
Usar solo el coste de esta regla	El porte de PrestaShop se ignora y se usa solo el de esta regla. Corta el bucle: ninguna regla posterior se evalúa.
Usar solo el coste básico	Se ignora esta regla y se deja el porte nativo de PrestaShop. Corta el bucle. Sirve como "regla de escape".

5. Fórmulas

Escribe una expresión matemática. **Se guarda solo si es válida** — si te equivocas, el módulo te avisa al guardar y no te deja continuar.

Reglas de escritura

- Decimales con **punto**, nunca coma: 1.5  · 1,5 
- Funciones **SIEMPRE EN MAYÚSCULAS**: ROUND(...)  · round(...) 
- Argumentos separados por **punto y coma**: MIN(10; 5)  · MIN(10, 5) 

Variables que puedes usar

Variable	Qué vale
total	Total del pedido, sin portes y sin IVA.
shipping	El coste de envío que había calculado PrestaShop.
quantity	Nº de unidades que cumplen las condiciones de la regla.
total_quantity	Nº de unidades totales del carrito.
nb_product	Nº de líneas del carrito seleccionadas.
weight	Peso.
width / height / depth / volume	Dimensiones de TODO el carrito.
weight_products / width_products / height_products / depth_products / volume_products	Ídem, pero solo de los productos seleccionados por la regla.
total_wholesale_price	Suma de los precios de coste.
total_exclude_promo	Total de los productos sin promoción.

 **Punto de confusión más común:** width, height, depth y volume **NO respetan** la selección de productos de la regla: son del carrito entero. Si quieres solo los productos seleccionados, usa las versiones terminadas en **_products**.

El desplegable "Variables and Functions" del formulario solo muestra 8 de las variables disponibles. Las demás de esta tabla también funcionan.

Funciones que puedes usar

ROUND(x) (redondea) · FLOOR(x) (redondea abajo) · CEIL(x) (redondea arriba) · MIN(x; y; ...) · MAX(x; y; ...) · ABS(x) · SQRT(x) · PRICE(x) (convierte a la moneda del cliente).

Ejemplos que funcionan

```
15 + (5 * (quantity - 1))    # 15 € el 1er producto, 5 € cada adicional
total * 0.05                 # 5 % del importe del pedido
CEIL(weight) * 1.20          # 1,20 € por kilo empezado
MAX(4.90; total * 0.03)      # 3 % del pedido, con un mínimo de 4,90 €
MIN(total * 0.10; 25)        # 10 % del pedido, con un tope de 25 €
shipping + (nb_product * 0.50) # 0,50 € por línea sobre el porte de PrestaShop
```

Para reglas de tipo **Weight**:

```
weight * 1.15                # sube el peso un 15 %
quantity * 0.3                # 0,3 kg de embalaje por unidad
0.5                           # suma 0,5 kg fijos
```

Errores típicos

```
round(total)      ✗ minúsculas – las funciones van en MAYÚSCULAS
MIN(10, 5)        ✗ coma – usa punto y coma: MIN(10; 5)
ROUND(1.234; 2)   ✗ ROUND solo acepta un argumento
total * 0,05      ✗ coma decimal – usa punto: total * 0.05
15 + (5 * quantity ✗ falta cerrar el paréntesis
```

⚠ **Evita los números negativos dentro de funciones:** `MIN(10; -5)` da un resultado incorrecto sin avisar. Reescríbelo como `MIN(10; 0 - 5)`.

6. La prioridad: lo que más equivocaciones causa

Las reglas se evalúan **de menor a mayor prioridad** (1, luego 2, luego 3...).

Tres tipos de regla **cortan el bucle** en cuanto se aplican, y las reglas posteriores **ni se miran**:

Regla	Efecto
Free shipping	Porte = 0 y STOP .
Usar solo el coste de esta regla	Porte = el de esta regla y STOP .
Usar solo el coste básico	Porte = el de PrestaShop y STOP .

Las de **"Sumar al coste básico"** son las únicas que **acumulan y siguen**.

Consecuencia práctica: si tienes una regla de *"Envío gratis a partir de 50 €"* con prioridad 5 y un *"Recargo Baleares"* con prioridad 10, un cliente de Baleares que gaste 60 € tendrá **envío gratis** — el recargo nunca llega a evaluarse. Si lo que quieres es que el recargo de Baleares se cobre igualmente, **dale prioridad 1** (antes que el envío gratis) y ponlo como *"Sumar al coste básico"*... pero ten en cuenta que el envío gratis lo pondrá a 0 igualmente. En ese caso, lo correcto es **añadir una condición de CP a la regla de envío gratis** para excluir Baleares.

7. Recetas listas para copiar

Objetivo	Information	Conditions	Actions
Envío gratis a partir de 50 €	"Envío gratis >50", prioridad 1	✓ Importe mínimo = 50, EUR, Impuestos incluidos , base = Total del pedido	Tipo = Free shipping
Recargo de 15 € a Baleares	"Recargo Baleares", prioridad 1	✓ Zip codes → grupo → país España , operador empieza por , valor 07	Tipo = Amount , 15 EUR, tu grupo de IVA de envíos, Compatibilidad = Sumar al coste básico
Ocultar el transportista urgente después de las 14:00	"Urgente solo mañanas"	✓ Hora: 00:00 → 14:00 · ✓ Carriers: selecciona el urgente	Tipo = Show only selected carriers
Suplemento por producto voluminoso	"Suplemento voluminoso"	✓ Dimensions → base <i>Por producto</i> , tipo <i>Combinada con contorno</i> , operador > , valor 300 (cm)	Tipo = Amount , 25 · Aplicar si = <i>Al menos un producto cumple</i>
Tarifa por peso volumétrico de la agencia	"Tarifa volumétrica"	✓ Package dimensions → ratio predefinido 166.667 kg/m3 · tramos: 0–5 kg → 6,50 € · 5–10 kg → 9,00 €	Compatibilidad = Usar solo el coste de esta regla
15 € el primer producto, 5 € cada adicional	"Escalado por unidades"	(sin restricciones, o las que necesites)	Tipo = Formula , 15 + (5 * (quantity - 1)) · Compatibilidad = Usar solo el coste de esta regla

8. Cosas que conviene saber (para no llevarse sorpresas)

1. **El módulo no funciona sin overrides activados.** (Rendimiento → "Desactivar todos los overrides" = **No**.)
2. **Editar y guardar una regla reconstruye TODAS sus condiciones desde cero.** Si una condición desaparece después de guardar, es porque su casilla quedó desmarcada.
3. **La ventana horaria usa la hora del SERVIDOR**, no la del cliente ni la de tu zona horaria si son distintas.
4. **La regla de tipo "Weight" no cambia el precio directamente:** cambia el peso que PrestaShop usa para elegir la tarifa. Si tu transportista no tiene tarifas por peso, no hará nada.
5. **"Formula" SOBREScribe, no suma.** Si en la misma regla combinas Formula con Percent o Amount, la fórmula **anula** lo demás. Usa un solo tipo por regla.
6. **"Aplicar a → Productos seleccionados"** solo tiene sentido con tipo **Percent** y requiere que hayas configurado condiciones de producto.
7. **Si defines solo el "número mínimo de proveedores" y no el máximo, la condición no hace nada.** Rellena siempre los dos.
8. **Desinstalar el módulo BORRA TODAS TUS REGLAS de forma irreversible.** No hay opción de conservar los datos. **Haz copia de seguridad de la base de datos antes de desinstalar.**
9. **El módulo no muestra nada al cliente final.** Solo modifica el importe de los portes.

9. Algo no cuadra: ¿por qué este pedido tiene ese coste de envío?

Usa el Simulador

Transporte → "Simulador de coste de envío" → "Un pedido existente"

Metes el **ID del pedido**, pulsas "**Cargar pedido**" y luego "**Ejecutar simulación**". Te dirá:

- Los portes que se **cobraron realmente** en el pedido.
- Los portes que las reglas **calculan hoy**.
- **Y regla por regla, en orden de prioridad:** si aplicó o no, qué hizo (sumó / substituyó / envió gratis / ocultó el transportista / alteró el peso), cuánto aportó, y **si no aplicó, exactamente qué condición falló** ("la regla exige código postal que empiece por 07 / el carrito tiene 28001").

También te mostrará los transportistas que **una regla ha ocultado** del checkout, que es la respuesta a "*¿por qué no me sale el transportista X?*".

Guía completa: [DOC_SIMULADOR.md](#).

⚠ **Dos avisos que el simulador te dará en rojo:**

1. Si has **editado alguna regla después** de que se hiciera el pedido, la simulación reproduce lo que las reglas harían **hoy**, no lo que pasó entonces. Una diferencia **no demuestra** que haya un fallo.
2. La hora, el tipo de cambio, el stock y los precios son **los de ahora**. Las reglas con franja horaria o con conversión de divisa no se pueden reproducir tal como estaban.

Pistas rápidas, sin abrir nada

Lo que ves	Probable causa
El porte es exactamente 0	Hay una regla de Free shipping que se cumplió, o el transportista está marcado como gratuito.
El porte es un importe redondo que no se parece al de PrestaShop	Una regla con "Usar solo el coste de esta regla" lo sustituyó.
El porte es el de PrestaShop + algo	Una o varias reglas de "Sumar al coste básico" se acumularon.
Faltaba un transportista en el checkout	Una regla de "Show only selected carriers" lo ocultó.
El porte cambia y no debería	Puede haber una regla de tipo Weight alterando el peso y, con él, el tramo de tarifa.
Mi regla no se aplica y debería	Lo más habitual: una regla de prioridad menor cortó el bucle antes de llegar a la tuya (envío gratis, "usar solo el coste de esta regla" o "usar solo el coste básico"). El simulador te lo marca con una fila azul.

Para pedidos muy antiguos

El módulo **no guarda** qué regla se aplicó a cada pedido, así que el simulador reconstruye el cálculo con la configuración actual. Si necesitas reconstruir un pedido antiguo a mano, con consultas SQL, el procedimiento está en [DOC_DESARROLLO.md §3](#).

orderfees_shipping — Documentación Técnica

Módulo: `orderfees_shipping` — "Advanced Shipping cost" **Autor:** motionSeed (módulo comercial de terceros, licencia propietaria) **Versión analizada:** 1.10.12 **Clase principal:** `OrderFees_Shipping` `extends MotionSeedModule` **module_key:** `3f8ccdf1109e18c34cc5f2d038c418dc` **Fecha del análisis:** 2026-07-14

1. Qué hace

`orderfees_shipping` es un **motor de reglas de gastos de envío**. Conceptualmente es un clon de `CartRule` (reglas de carrito) aplicado al transporte: el comerciante define reglas con N condiciones (país, zona, provincia, ciudad, código postal, grupo de cliente, transportista, productos/categorías/atributos, dimensiones, peso, importe del carrito, franja horaria, nº de proveedores) y, cuando una regla se cumple contra el carrito, produce un **valor** (importe fijo, porcentaje, fórmula matemática, envío gratis, tarifa por tramos de peso volumétrico) que **suma, sustituye o anula** el coste de transporte nativo de PrestaShop.

Técnicamente lo consigue **overrideando** `Cart::getPackageShippingCost()` y `Cart::getTotalWeight()` y disparando dos hooks propios que el propio módulo escucha.

Casos de uso cubiertos (verificados en código):

- Envío gratis a partir de X € (`type & IS_FREE_SHIPPING`).
 - Recargo geográfico (Baleares/Canarias/Ceuta) por CP con operador *empieza por*.
 - **Ocultar un transportista** en el checkout si no se cumplen condiciones (`type & IS_CARRIER`).
 - Tarifa por **peso volumétrico** con ratios de agencia (333 / 250 / 200 / 166,667 / 150 / 142,857 kg/m³, o 5000 / 6000 / 7000 cm³/kg) y baremo por tramos.
 - Precio escalado por unidades vía fórmula: `15 + (5 * (quantity - 1))` .
 - Suplemento por producto voluminoso (`combined_girth = 2A + 2Al + P > X cm`).
 - **Modificar el peso del carrito** con una fórmula (`IS_WEIGHT`) para alterar la fila de `ps_delivery` que PrestaShop elige.
 - Recargo por multi-proveedor (`nb_supplier_min` / `nb_supplier_max`).
 - Reglas con ventana horaria, por cliente concreto, por grupo, por tratamiento (Sr./Sra.) y por tienda.
-

2. Arquitectura

2.1 Ficheros clave

Fichero	Rol
orderfees_shipping.php (421 líneas)	Clase principal. Motor del bucle de reglas: <code>hookActionCartGetPackageShippingCost()</code> (:105-328) y <code>hookActionCartGetTotalWeight()</code> (:330-372).
classes/ShippingRule.php (1947 líneas)	ObjectModel central. <code>check()</code> (:680-1186) evalúa condiciones; <code>value()</code> (:1188-1383) calcula el importe; <code>restrictionsProducts()</code> (:1565-1766) es el motor de condiciones de producto.
override/classes/Cart.php	Override base (PS 1.6.0 – 1.7.6.x). 2 métodos, ~60 líneas, cero lógica de negocio .
specs/1.7.7.1/override/classes/Cart.php	Override para PS ≥ 1.7.7.1 (incluye PS 8 y PS 9) . Única diferencia: parámetro <code>bool \$keepOrderPrices = false</code> .
helpers/motionseed-module/MotionSeedModule.php (803 líneas)	Framework interno del vendor: autoload, install/uninstall, overrides por versión (<code>specificOverride()</code> :332-376), menús, traducciones legacy.
controllers/admin/AdminOrderFeesShippingController.php (1799 líneas)	CRUD de reglas + AJAX de los constructores dinámicos.
classes/phpmathparser/	Parser matemático embebido (PHP Math Parser de ircmaxell, MIT) + 2 clases propias motionSeed.
config/sql/install.sql	21 tablas (<code>CREATE TABLE IF NOT EXISTS</code>).
config/menus.xml	Una entrada: <code><menu name="Advanced Shipping cost" parent="AdminParentShipping" class="AdminOrderFeesShipping" /></code> .
views/js/admin.js	Ofuscado (javascript-obfuscator). Pestañas, constructores AJAX, dobles listas.
upgrade/	13 <code>upgrade-*.php</code> , 9 <code>upgrade.sql</code> , 2 snapshots de override.

NO existe `override/classes/Carrier.php`. El único override es `Cart`.

2.2 Flujo de ejecución completo

```

1. Front: el cliente selecciona transportista en el checkout
↓
2. CartCore::getDeliveryOptionList() → por cada paquete × cada transportista:
↓
3. Cart::getPackageShippingCost() ← OVERRIDE del módulo (override/classes/Cart.php:13)
↓
4. Inicializa $total = 0; $return = false; (Cart.php:20-21)
↓
5. Hook::exec('actionCartGetPackageShippingCost', [...POR REFERENCIA...]) (Cart.php:23-33)
← se dispara ANTES de llamar a parent::
↓
6. OrderFees_Shipping::hookActionCartGetPackageShippingCost(&$params) (orderfees_shipping.php:105)
|
| :107-109 Guard anti-recursión: if (self::$disable_calculation) return;
| :111 self::$weight_rules = [] (buffer para el hook de peso)
| :115-126 SELECT reglas activas + GROUP_CONCAT de carriers
| LEFT JOIN carrier c ON c.id_reference = src.id_carrier AND c.deleted = 0
| WHERE sr.active = 1
| ORDER BY sr.priority ASC ← ORDEN DE EVALUACIÓN
| :128-130 Si no hay reglas → return (comportamiento nativo intacto)
| :136-148 Resuelve la DIRECCIÓN (PS_TAX_ADDRESS_TYPE; si no, id_address_delivery
| del PRIMER producto del carrito). Valida con Address::addressExists()
| :150-155 Resuelve el CARRIER (si vacío → PS_CARRIER_DEFAULT)
|
| :159-327 BUCLE POR REGLA (priority ASC)
| | :162-164 skip si está en $items_disabled (anulada por combinación)
| | :166 ShippingRule::factory($id, $cart) ← inyecta el carrito
| | :168-172 disable_calculation = true → $rule->check() → false
| | :174-176 si !check() → continue
| | :178-191 FILTRO DE TRANSPORTISTA
| | · carrier_restriction && (type & IS_CARRIER) →
| | total = false; return = true; RETURN ⇒ TRANSPORTISTA OCULTO
| | · lista no vacía y el carrier no está → continue
| | · lista vacía → la regla aplica a todos
| | :193-217 COMBINACIONES: las reglas no emparejadas se añaden a $items_disabled
| | :219-221 type == IS_NONE (0) y sin package_restriction → continue
| | :223-227 type == IS_WEIGHT (1024) → self::$weight_rules[] = regla; continue
| | :229-234 type & IS_FREE_SHIPPING (1) → total = 0; return = true; RETURN ★STOP
| | :236-305 package_restriction → tarifa por tramos de peso volumétrico ($4.4)
| | :306-312 resto → $total = ps_round($rule->value($params), _PS_PRICE_COMPUTE_PRECISION_)
| | :314-326 DECISIÓN TERMINAL (bitmask)
| | · BASIC_SHIPPING_RULE (64) → total = $total; return = true; RETURN ★STOP (SUSTITUYE)
| | · BASIC_SHIPPING_BASE (128) → total = 0; return = false; RETURN ★STOP (SOLO PS)
| | · else (BASIC_SHIPPING_MERGE 32, implícito)
| | → $params['total'] += $total; ← ACUMULA y sigue con la siguiente regla
|
↓
7. Vuelta al override:
· if ($return) → return ($total !== false ? ps_round($total, 2) : false); (Cart.php:35)
· else → $shipping_cost = parent::getPackageShippingCost(...); (Cart.php:38)
return $shipping_cost + ps_round($total, 2); (Cart.php:47)
↓
8. En paralelo, cuando PrestaShop pide el peso:
Cart::getTotalWeight() ← OVERRIDE (Cart.php:53-70)
→ Hook::exec('actionCartGetTotalWeight')
→ OrderFees_Shipping::hookActionCartGetTotalWeight() (orderfees_shipping.php:330)
· si self::$weight_rules está vacío → return
· por cada regla de peso: ShippingRule::parse($rule->formula, $data)
→ $params['total_weight'] += (float)$result
→ return parent::getTotalWeight($products) + $total_weight;
⇒ altera el rango de peso que PS usa para elegir la fila de ps_delivery
↓
9. El importe se pinta en el carrito/checkout y, al validar, se congela en
ps_orders.total_shipping y ps_order_carrier.shipping_cost_tax_*
```

Guard anti-recursión (crítico): `self::$disable_calculation` (orderfees_shipping.php:39) se activa alrededor de `check()` (:168-172) y de `value()` (:307-311), porque ambos llaman a `$cart->getOrderTotal()`, que volvería a invocar `getPackageShippingCost()` → override → hook → recursión infinita.

Instalación del override: `MotionSeedModule::specificOverride()` (MotionSeedModule.php:332-376) escanea `specs/*`, elige la carpeta de mayor versión ≤ `_PS_VERSION_` e intercambia temporalmente `local_path`. Como solo existe `specs/1.7.7.1`, cualquier PS ≥ 1.7.7.1 (incluidos PS 8 y PS 9) instala la variante con `$keepOrderPrices`.

Dependencia oculta con el módulo hermano `orderfees`: `OrderFees_Shipping::specificOverride()` (`orderfees_shipping.php:83-90`) — si `Module::isEnabled('orderfees')` , entonces `$this->disabled_overrides = array('Cart')` ⇒ **este módulo NO instala su override de Cart** y depende al 100 % de que el override de `orderfees` dispare los dos hooks.

2.3 Hooks

Registrados (`orderfees_shipping.php:97-103` + `MotionSeedModule.php:269-272`): `actionCartGetPackageShippingCost` · `actionCartGetTotalWeight` · `actionObjectLanguageAddAfter` · `actionDiscoverModules` .

Implementado pero NO registrado: `hookActionCartSummary()` (`orderfees_shipping.php:374-387`) — la línea de registro está comentada (`:102`). Código muerto.

Hooks de extensión que el módulo dispara (para terceros): `actionShippingRuleCheck` (`ShippingRule.php:1171`), `actionShippingRuleValue` (`ShippingRule.php:1201`), `actionShippingRuleDuplicate` , `actionAdminOrderFeesShippingValidateBefore/After` , `actionAdminOrderFeesShippingRenderForm` , `actionAdminOrderFeesShippingAjaxProcess` .

3. Modelo de datos

21 tablas, todas `ENGINE=_MYSQL_ENGINE_ DEFAULT CHARSET=UTF8` , en `config/sql/install.sql` . Sin **FOREIGN KEYS** (integridad en PHP, patrón PrestaShop).

3.1 Tabla maestra PREFIX_of_shipping_rule (install.sql:2-46)

Columna	Tipo	Significado
id_of_shipping_rule	INT UNSIGNED AI, PK	Identificador de la regla.
name	VARCHAR(255)	Nombre interno (admin). No es multilingüe — no hay tabla <code>_lang</code> .
id_customer	INT UNSIGNED, def 0	Restringe la regla a un cliente concreto . 0 = todos.
date_from / date_to	DATETIME NULL	Ventana de validez por fecha.
time_from / time_to	TIME NULL	Ventana horaria diaria . Comparadas contra <code>date('H:i:s')</code> del servidor .
priority	INT UNSIGNED, def 1	Orden de evaluación ASC . Menor número = se evalúa antes.
minimum_amount	DECIMAL(17,2)	Umbral mínimo. Comparación inclusiva (< min → falla).
minimum_amount_tax	TINYINT(1)	0 = sin IVA (total), 1 = con IVA (total_wt).
minimum_amount_currency	INT UNSIGNED	Moneda del umbral; se convierte a la del carrito.
minimum_amount_restriction	INT	Base de cálculo : 0 = desactivado · 1 = ORDER (total sin envío) · 2 = PRODUCTS (solo los productos seleccionados) · 4 = SHIPPING (solo portes).
maximum_amount + _tax + _currency + _restriction	idem	Espejo. Comparación exclusiva (>= max → falla).
country_restriction	TINYINT(1)	Flag → tabla <code>_country</code> .
zone_restriction	TINYINT(1)	Flag → <code>_zone</code> .
state_restriction	TINYINT(1)	Flag → <code>_state</code> .
city_restriction	TINYINT(1)	Flag → <code>_city</code> (lista plana) o <code>_city_rule_group</code> / <code>_city_rule</code> .
carrier_restriction	TINYINT(1)	Flag → <code>_carrier</code> . NO se evalúa en check() — lo aplica el hook (<code>orderfees_shipping.php:178-191</code>).
group_restriction	TINYINT(1)	Flag → <code>_group</code> (grupos de cliente).
of_shipping_rule_restriction	TINYINT(1)	Exclusividad : si 1, solo combinable con las reglas de <code>_combination</code> .
product_restriction	TINYINT(1)	Activa el árbol <code>_product_rule_group</code> → <code>_product_rule</code> → <code>_product_rule_value</code> .
shop_restriction	TINYINT(1)	Multitienda → <code>_shop</code> .
dimension_restriction	TINYINT(1)	Activa <code>_dimension_rule_group</code> → <code>_dimension_rule</code> .
zipcode_restriction	TINYINT(1)	Activa <code>_zipcode_rule_group</code> → <code>_zipcode_rule</code> .
package_restriction	TINYINT(1)	NO es condición, es ACCIÓN : activa el baremo <code>_package_rule_group</code> → <code>_package_rule</code> que reemplaza a <code>value()</code> .
gender_restriction	TINYINT(1)	Flag → <code>_gender</code> (tratamiento Sr./Sra.).
nb_supplier_min / nb_supplier_max	INT UNSIGNED NULL	Nº de proveedores distintos en el carrito.
type	INT UNSIGNED	Máscara de bits (acción + modo de fusión + modo de aplicación). Ver §4.1.
percent	DECIMAL(5,2)	Porcentaje si <code>type & IS_PERCENT</code> . Admite negativos (-100...100).
amount	DECIMAL(17,2)	Importe fijo si <code>type & IS_AMOUNT</code> .
formula	VARCHAR(255) NULL	Expresión matemática si <code>type & IS_FORMULA</code> o <code>type == IS_WEIGHT</code> .

Columna	Tipo	Significado
<code>currency</code>	INT UNSIGNED	Moneda del <code>amount</code> ; se convierte vía <code>conversion_rate</code> .
<code>tax_rules_group</code>	INT	Grupo de reglas de impuestos aplicado al valor calculado. <code>0</code> = sin IVA.
<code>product</code>	INT	Ámbito del %: <code>0</code> = total del carrito · <code>>0</code> = un <code>id_product</code> concreto · <code>-2</code> (<code>PERCENT_PRODUCTS</code>) = la selección de las product rules.
<code>quantity_per_product</code>	TINYINT(1)	Si 1, el valor se multiplica por la cantidad de unidades seleccionadas. UI oculto.
<code>active</code>	TINYINT(1)	Regla habilitada.
<code>date_add</code> / <code>date_upd</code>	DATETIME	Auditoría estándar.

Índices: **solo PRIMARY** . Los índices `id_customer` y `group_restriction` se eliminaron en 1.8.5.

3.2 Satélites de restricción simples (PK compuesta, M:N)

Tabla	Columnas	Notas
<code>_carrier</code> (:49-53)	<code>id_of_shipping_rule</code> , <code>id_carrier</code>	⚠ <code>id_carrier</code> guarda en realidad el <code>id_reference</code> del transportista. Ver el JOIN <code>c.id_reference = src.id_carrier</code> (<code>orderfees_shipping.php:121</code>). Es así para que la regla sobreviva a la edición del transportista (PS crea un carrier nuevo con la misma <code>id_reference</code>). Error nº 1 al depurar a mano.
<code>_country</code> (:65-69)	<code>id_of_shipping_rule</code> , <code>id_country</code>	País de la dirección de entrega.
<code>_zone</code> (:72-76)	<code>id_of_shipping_rule</code> , <code>id_zone</code>	
<code>_state</code> (:79-83)	<code>id_of_shipping_rule</code> , <code>id_state</code>	
<code>_group</code> (:93-97)	<code>id_of_shipping_rule</code> , <code>id_group</code>	Invitado → <code>PS_UNIDENTIFIED_GROUP</code> .
<code>_shop</code> (:123-127)	<code>id_of_shipping_rule</code> , <code>id_shop</code>	
<code>_gender</code> (:205-209)	<code>id_of_shipping_rule</code> , <code>id_gender</code>	
<code>_city</code> (:86-90)	<code>id_of_shipping_rule</code> , <code>name VARCHAR(255)</code>	Ciudad como texto (PS no modela ciudades). Solo en el modo <code>cityselect</code> .
<code>_combination</code> (:56-62)	<code>id_of_shipping_rule_1</code> , <code>id_of_shipping_rule_2</code>	Autorreferencia simétrica (se consulta con OR en ambos sentidos). Semántica: compatibilidad entre reglas.

3.3 Árbol de productos (3 niveles)

Tabla	Columnas
<code>_product_rule_group</code> (:108-113)	<code>id_product_rule_group</code> (AI, PK), <code>id_of_shipping_rule</code> , <code>quantity</code> = nº mínimo de unidades que deben cumplir el grupo.
<code>_product_rule</code> (:100-105)	<code>id_product_rule</code> (AI, PK), <code>id_product_rule_group</code> , <code>type</code> VARCHAR(32) ∈ <code>products \ categories \ attributes \ features \ manufacturers \ suppliers \ main_categories \ outofstock</code> .
<code>_product_rule_value</code> (:116-120)	<code>id_product_rule</code> , <code>id_item</code> = ID polimórfico (su tabla destino depende de <code>product_rule.type</code>).

3.4 Árboles de 2 niveles (valores en CSV en la columna value)

Tabla grupo	Tabla regla	Columnas de la regla
<code>_dimension_rule_group (:130-135) — base</code> <code>∈ product / product_quantity / all</code>	<code>_dimension_rule</code> (:138-145)	<code>type ∈ width, height, depth, weight, volume, combined,</code> <code>combined_girth · operator ∈ = > < >= <= != · value TEXT (CSV).</code>
<code>_city_rule_group (:148-152)</code>	<code>_city_rule (:155-161)</code>	⚠ type NO es un tipo: es el id_country de ámbito ('' = cualquiera) · value TEXT (CSV de ciudades). Sin operador.
<code>_zipcode_rule_group (:164-168)</code>	<code>_zipcode_rule</code> (:171-178)	<code>type = id_country de ámbito · operator ∈ = > < >= <= != begin end</code> <code>· value TEXT (CSV de CPs).</code>
<code>_package_rule_group (:181-188) — unit ∈</code> <code>kg/m3 cm3/kg, unit_weight, ratio</code>	<code>_package_rule</code> (:191-202)	<code>range_start, range_end (tramo de peso facturable) · round (múltiplo de</code> <code>redondeo) · divider (precio por cada N unidades) · currency · tax (0/1) ·</code> <code>value (importe del tramo).</code>

3.5 Relaciones (todas lógicas, sin FK)

```

of_shipping_rule (1) ←< _carrier      → ps_carrier.id_reference ⚠
(1) ←< _country      → ps_country.id_country
(1) ←< _zone          → ps_zone.id_zone
(1) ←< _state         → ps_state.id_state
(1) ←< _group         → ps_group.id_group
(1) ←< _shop          → ps_shop.id_shop
(1) ←< _gender        → ps_gender.id_gender
(1) ←< _city          (name = texto libre)
(N) ←< _combination >— (N) of_shipping_rule [simétrica]
(1) ←< _product_rule_group ←< _product_rule ←< _product_rule_value
(1) ←< _dimension_rule_group ←< _dimension_rule
(1) ←< _city_rule_group   ←< _city_rule      (type → ps_country)
(1) ←< _zipcode_rule_group ←< _zipcode_rule  (type → ps_country)
(1) ←< _package_rule_group ←< _package_rule

```

Escalares: `id_customer → ps_customer · {minimum,maximum}_amount_currency, currency → ps_currency`
`tax_rules_group → ps_tax_rules_group · product → ps_product (0 = carrito, -2 = selección)`

Borrado en cascada manual: `ShippingRule::delete()` (`ShippingRule.php:158-279`) purga 19 tablas hijas. **NO purga** `of_shipping_rule_city` →
filas huérfanas.

3.6 Historial de esquema (upgrades relevantes)

Versión	Cambio
1.8.4	+ <code>time_from</code> , + <code>time_to</code> → ventanas horarias.
1.8.5	<code>formula</code> y <code>date_from/to</code> pasan a NULL; DROP INDEX <code>id_customer</code> , DROP INDEX <code>group_restriction</code> .
1.8.9	+ <code>zone_restriction</code> + tabla <code>_zone</code> .
1.8.13	RENAME <code>minimum_amount_shipping</code> → <code>minimum_amount_restriction</code> (bool → enumerado).
1.8.14	+ <code>state_restriction</code> , + <code>gender_restriction</code> + tablas <code>_state</code> , <code>_gender</code> .
1.8.15	Crea <code>MS_ORDERFEES_SHIPPING_TYPES</code> (solo si vacía).
1.8.17	+ <code>nb_supplier_min</code> , + <code>nb_supplier_max</code> .
1.8.24	<code>upgradeOverride('Cart') : getPackageShippingCost()</code> pasa a <code>return (\$total !== false ? ps_round(...) : false)</code> → nace la feature "ocultar transportista" .
1.8.26	Sobrescribe <code>MS_ORDERFEES_SHIPPING_TYPES</code> incondicionalmente = <code>country,zone,state,carrier,group,of_shipping_rule,shop</code> .
1.8.28	<code>dimension_rule.value</code> y <code>zipcode_rule.value</code> → TEXT (las listas largas de CPs se truncaban).
1.8.30	UPDATE <code>of_shipping_rule</code> SET <code>type = 513</code> WHERE <code>type = 1</code> (513 = 512 APPLY_IF_ALL + 1 IS_FREE_SHIPPING).
1.9.3	<code>upgradeOverride('Cart')</code> + <code>registerHook('actionCartGetTotalWeight')</code> → nacen las reglas de peso (IS_WEIGHT) .
1.9.4	+ <code>city_restriction</code> + 3 tablas de ciudad; <code>product_rule.type</code> → VARCHAR(32).
1.10.6	UPDATE ... <code>minimum_amount_restriction = IF(minimum_amount = 0, 0, minimum_amount_restriction + 1)</code> (idem max). Recodifica el enumerado. Migración de datos NO idempotente.
1.10.7 – 1.10.12	Sin ficheros de upgrade → solo cambios PHP/TPL.

4. Motor de reglas

4.1 Bitmask type (ShippingRule.php:112-131)

Grupo	Constante	Valor	Semántica
Acción	IS_NONE	0	Sin acción. El hook la salta salvo que tenga <code>package_restriction</code> .
	IS_FREE_SHIPPING	1	Envío gratis. <code>total = 0; return = true</code> → STOP total .
	IS_PERCENT	2	Porcentaje sobre el ámbito de <code>product</code> . Acumula (<code>\$value +=</code>).
	IS_AMOUNT	4	Importe fijo, con conversión de moneda. Acumula (<code>\$value +=</code>).
	IS_FORMULA	8	Fórmula del math parser. ⚠ SOBREScribe (<code>\$value = \$result;</code>), NO suma.
	IS_CARRIER	16	Veta el transportista si no está en la lista → <code>total = false; return = true</code> .
	IS_WEIGHT	1024	No calcula precio: modifica el peso del carrito. Se comprueba con <code>==</code> estricto , no bit a bit.
Fusión con el porte base	BASIC_SHIPPING_MERGE	32	Sumar al envío base. Es el else implícito — el bit nunca se testea.
	BASIC_SHIPPING_RULE	64	Sustituir : <code>total = \$total; return = true</code> → STOP .
	BASIC_SHIPPING_BASE	128	Ignorar la regla : <code>total = 0; return = false</code> → STOP , deja el porte nativo.
Aplicación multi-producto	APPLY_IF_LEAST	256	Basta con que un producto cumpla. Modo por defecto — el bit nunca se testea.
	APPLY_IF_ALL	512	Todos los productos deben cumplir. Se testea en dimensiones (:995-1001) y product rules (:1740-1742).
Ámbito del %	PERCENT_PRODUCTS	-2	Valor sentinela guardado en la columna <code>product</code> .
Base de importes	ORDER / PRODUCTS / SHIPPING	1 / 2 / 4	Base del umbral min/max.

4.2 Condiciones — ShippingRule::check() (:680-1186)

Devuelve `true` **solo si TODAS las condiciones activas se cumplen** (AND global; cualquier fallo → `return false` inmediato — hay **21** `return false` distintos). Requiere `$this->cart` inyectado por `factory()` (:1866-1873): un `new ShippingRule($id)` directo hace petar `check()` en :684.

Orden exacto de evaluación:

#	Condición	Líneas	Semántica
1	Activa / fechas / cliente / franja horaria	:687-695	<code>!active OR date_from > now OR date_to < now OR (id_customer ≠ el del carrito) OR fuera de time_from – time_to (hora del servidor)</code> → false.
2	Importe mínimo	:698-731	Convierte el umbral a la moneda del carrito. Base según <code>minimum_amount_restriction : PRODUCTS</code> → suma de los productos seleccionados; <code>SHIPPING</code> → <code>getOrderTotal(tax, Cart::ONLY_SHIPPING)</code> ; <code>ORDER</code> → <code>getOrderTotal(tax, Cart::BOTH_WITHOUT_SHIPPING)</code> . Falla si <code>cart_total < minimum_amount (inclusivo)</code> .
3	Importe máximo	:734-767	Idéntico. Falla si <code>cart_total >= maximum_amount (exclusivo)</code> .
4	Ciudad	:770-821	Modo A: módulo <code>cityselect</code> activo y sin city rule groups → match exacto contra <code>_city.name</code> . Modo B: normaliza con <code>formatCity()</code> (:1861-1864 : quita todo lo que no sea letra Unicode + minúsculas) y evalúa grupos. AND entre grupos, OR dentro.
5	Código postal	:824-892	Normaliza a minúsculas + trim. Operadores: <code>!=</code> , <code>begin (strpos(\$postcode, \$value) === 0), end</code> , <code>y =</code> , <code>></code> , <code><</code> , <code>>=</code> , <code><=</code> como comparación de STRINGS , NO numérica. AND entre grupos, OR dentro.
6	Dimensiones	:895-1008	Ver §4.3.
7	Productos	:1011-1013 → :1565	<code>restrictionsProducts()</code> . Ver §4.5.
8	Grupo de cliente	:1016-1033	Invitado → <code>PS_UNIDENTIFIED_GROUP</code> .
9	País	:1036-1065	Dirección de entrega; fallback <code>context->country->id</code> .
10	Zona	:1068-1099	Resuelve <code>country.id_zone</code> vía JOIN.
11	Estado/provincia	:1102-1122	Requiere dirección de entrega.
12	Tienda	:1125-1136	Solo si <code>Shop::isFeatureActive()</code> .
13	Tratamiento/género	:1139-1155	Con invitado (<code>id_customer = 0</code>) el subselect da NULL → false.
14	Nº de proveedores	:1158-1169	<code>COUNT(DISTINCT ps.id_supplier)</code> sobre <code>cart_product × product_supplier</code> . ⚠ Solo se evalúa si nb_supplier_max es truthy.
15	Hook <code>actionShippingRuleCheck</code>	:1171-1183	Si cualquier módulo devuelve un valor no-null y falsy → false.

NO se evalúan en `check()`: `carrier_restriction` (lo hace el hook) ni `package_restriction` (es una acción de cálculo).

Semántica AND/OR global:

- Entre bloques de restricción distintos → **AND estricto**.
- Entre grupos de reglas (product/dimension/city/zipcode) → **AND**.
- Dentro de un grupo → **OR**.
- Entre los valores CSV de una regla → **OR**.
- `APPLY_IF_ALL` (512) invierte la semántica **a nivel de producto** en dimensiones y product rules.

4.3 Motor de dimensiones (:895-1008)

Precalcula tres bases:

base	Contenido	Semántica
product	Valor unitario de cada producto	"cada producto individual mide X"
product_quantity	max() de dim × cart_quantity	"el producto que más aporta"
all	SUM(dim × cart_quantity)	"el carrito entero mide X"

Dimensiones (type): width , height , depth , weight ; volume = h*w*d ; combined = h+w+d ; combined_girth = 2h + 2w + d (fórmula típica de UPS/FedEx). Operadores: = > < >= <= !=.

⚠ Si **ningún producto tiene dimensiones** (:960), la restricción **se ignora silenciosamente** y la regla pasa.

4.4 Motor de "package" (tarifa por tramos de peso volumétrico) — orderfees_shipping.php:236-305

Aunque el flag se llama package_restriction , **no es una condición: es un método de cálculo alternativo que REEMPLAZA a value()**.

- Suma weight += product.weight * cart_quantity y volume += (h*w*d) * cart_quantity sobre los productos seleccionados por la regla.
- Peso volumétrico según unit del grupo: - kg/m3 → volumeTo(volume, 'm3') * ratio - cm3/kg → volumeTo(volume, 'cm3') / ratio
- Peso facturable** = max(peso_volumétrico, weightTo(peso_real, 'kg')) .
- Busca el tramo cuyo CEIL(weight / round) * round cae BETWEEN range_start AND range_end .
- Precio: si divider = 1 → value (precio fijo del tramo); si no → ((CEIL(weight/round) * round) * value) / divider .
- Aplica el IVA **del transportista** (no el tax_rules_group de la regla) si use_tax && !Tax::excludeTaxOption() . Si PS_ATCP_SHIPWRAP está activo → carrier_tax = 0 .
- ps_round con los decimales de la moneda del tramo.

⚠ \$total se **SOBREScribe** (no acumula) en el bucle de grupos (:299): con varios package_rule_group solo prevalece el último.

Conversiones: \$WEIGHT_UNITS (:20-26 : g=0.001, kg=1.0, T=1000, lb=0.45359237, kip=453.59237) y \$VOLUME_UNITS (:28-34 : m3=1.0, cm3=1e-6, in3=0.000016, ft3=0.028317, yd3=0.764555).

4.5 Motor de condiciones de producto — restrictionsProducts() (:1565-1766)

Doble propósito: \$return_products = false → bool ; = true → array de claves "id_product-id_product_attribute" seleccionadas.

type	Consulta	Match
features	cart_product × feature_product	id_feature_value ∈ values. Clave id_product-0 .
attributes	cart_product × product_attribute_combination	id_attribute ∈ values.
products	cart_product	id_product ∈ values.
categories	cart_product × category_product	cualquier categoría asociada.
manufacturers	cart_product × product	id_manufacturer ∈ values.
suppliers	cart_product × product	⚠ p.id_supplier (el default del producto, NO product_supplier).
main_categories	cart_product × product	id_category_default ∈ values.
outofstock	StockAvailable::getQuantityAvailableByProduct()	Hack (:1720): \$count_matching_products = \$product_rule_group['quantity'] → siempre pasa el umbral.

Semántica resultante (contraintuitiva):

- Sin product_restriction , "seleccionados" = array vacío = **TODOS**.
- Entre grupos: AND para "¿aplica?", OR (unión) para "¿qué productos selecciona?".
- Dentro de un grupo con N > 1 reglas: OR para el umbral de cantidad, pero **AND (intersección)** para el conjunto de productos → comportamiento **mixto**.
- arrayUIntersectCompare() (:1889-1902): "5-0" hace match con "5-42" — una regla "por producto" captura todas sus combinaciones.

4.6 Cálculo del valor — ShippingRule::value(\$params) (:1188-1383)

No existe ningún método calculate() . El único punto de cálculo es value() .

`$params` esperado: `['use_tax' => bool, 'total' => float]` (el `total` es el porte base de PS, disponible como variable `shipping`).

- **Caché** (:1195-1199 , :1380): clave `orderfees_shipping_{id}_{use_tax}_{id_cart}`. ⚠ **NO incluye `id_carrier` ni `$params['total']`**.
- **Hook** `actionShippingRuleValue` (:1201-1205).
- **IS_PERCENT** (:1207-1239): 3 variantes según `product`:
- `product == 0` → `getOrderTotal(false, BOTH_WITHOUT_SHIPPING) * percent/100` → **siempre SIN IVA** (hardcoded).
- `product > 0` → `$product['total_wt'] * percent/100` → **siempre CON IVA**. ⚠ Incoherente.
- `product == -2` → suma `total_wt` de los productos seleccionados × `percent/100`.
- **IS_AMOUNT** (:1242-1260): conversión de moneda manual. `$value +=`.
- **IS_FORMULA** (:1263-1348): construye las variables (\$6) y evalúa con `self::parse()`. ⚠ `$value = $result;` — **SOBREScribe**.
- **Impuestos** (:1350-1359): si `use_tax`, `$value *= 1 + getTaxesRate($this->tax_rules_group, new Address(...)) / 100`.
- **quantity_per_product** (:1361-1378): `$value *= $quantity` — **DESPUÉS de los impuestos**.

5. Fórmulas (parser matemático)

Librería embebida en `classes/phpmathparser/` (PHP Math Parser de Anthony Ferrara / ircmaxell, MIT) + 2 clases propias motionSeed. **Singleton estático** en `ShippingRule.php:133`.

Punto de entrada único: `ShippingRule::parse($formula, $data)` (:1922-1937). **Contrato de retorno:** `float` = OK · `string` = mensaje de error (no lanza excepción).

Sintaxis: - Números decimales con **punto**. La coma está prohibida. - Operadores: `+` `-` `*` `/` `%` (módulo ENTERO) `^` (potencia, **asociativa por la izquierda**) y paréntesis. - **Funciones SIEMPRE EN MAYÚSCULAS** (el tokenizador trata cualquier token `[a-z_]{2,}` como variable). - **Argumentos separados por ; (punto y coma)**, nunca coma.

Funciones: `ABS`, `COS`, `COSH`, `SIN`, `SINH`, `TAN`, `TANH`, `ACOS`, `ACOSH`, `ASIN`, `ASINH`, `ATAN`, `ATANH`, `RAD2DEG`, `CEIL`, `FLOOR`, `ROUND`, `SQRT`, `LOG10` (1 argumento) · `MIN(x;y;...)`, `MAX(x;y;...)`, `PRICE(x)` (variádicas). ⚠ `PI`, `ATAN2` y `DEG2GRAD` están declaradas pero **fatalizan**. `ROUND(x)` **no acepta decimales**.

Variables (canónicas en `AdminOrderFeesShippingController.php:274-294`):

Variable	Significado	Coste (IS_FORMULA)	Peso (IS_WEIGHT)
<code>total</code>	Total del pedido sin envío, sin IVA	✓	✗
<code>shipping</code>	⚠ NO es el porte nativo de PrestaShop . Es <code>\$params['total']</code> (<code>ShippingRule.php:1266</code>), el acumulador de las reglas anteriores , que el override inicializa a <code>0</code> (<code>override/classes/Cart.php:20</code>). Para la primera regla evaluada vale <code>0</code> .	✓	✗
<code>quantity</code>	Unidades que cumplen las restricciones de la regla	✓	✓
<code>total_quantity</code>	Unidades totales del carrito	✓	✗
<code>nb_product</code>	Nº de líneas de carrito seleccionadas	✓	✗
<code>nb_product_exclude_attributes</code>	Nº de <code>id_product</code> distintos seleccionados	✓	✗
<code>weight</code>	Peso total × cantidad. Sin restricción de producto → <code>\$cart->getTotalWeight()</code>	✓	✓
<code>width / height / depth / volume</code>	Σ de TODOS los productos (no respetan la selección)	✓	✗
<code>weight_products / width_products / height_products / depth_products / volume_products</code>	Ídem, solo los productos SELECCIONADOS	✓	✗
<code>total_wholesale_price</code>	Σ <code>wholesale_price</code> × <code>cart_quantity</code> de todo el carrito	✓	✗
<code>total_exclude_promo</code>	Σ <code>total</code> de los productos sin reducción	✓	✗
<code>total_wholesale_price_exclude_promo</code>	Ídem con <code>wholesale_price</code>	✓	✗

⚠ El desplegable "Variables and Functions" del back-office solo ofrece **8 de las 19 variables** y **5 de las funciones**. El resto funcionan pero no están documentadas en la UI.

Propagación de errores:

Punto	Comportamiento
<code>AdminOrderFeesShippingController.php:296-303</code>	Al guardar , evalúa con datos de prueba. Si devuelve string → error visible y la regla no se guarda. Es la única validación visible .
<code>ShippingRule::value()</code> :1345	Si falla, se ignora EN SILENCIO y el recargo queda a 0.
<code>orderfees_shipping.php:366-370</code>	Si falla, el peso extra no se aplica, EN SILENCIO .

6. Configuration keys

Key	Default	Uso
<code>MS_ORDERFEES_SHIPPING_CONDITIONS_DISPLAY_SKU</code>	1	Mostrar SKU en el selector de productos (<code>ShippingRule.php:1690</code>).
<code>MS_ORDERFEES_SHIPPING_TYPES</code>	country, zone, state, city, carrier, group, of_shipping_rule, shop en install ; upgrade-1.8.26 lo sobrescribe SIN city	Tipos de restricción activos (<code>ShippingRule.php:1434 isSupportedType()</code>).
<code>MS_ORDERFEES_SHIPPING_CITYSELECT</code>	NO declarada ni inicializada	Leída en <code>ShippingRule.php:1521</code> . Bug latente.

7. Trazabilidad: qué queda registrado

El motor del vendor no registra NADA.

- No hay carpeta `/logs` , ni flag de debug, ni `PrestaShopLogger::addLog()` .
- Las 21 tablas del vendor son **todas de configuración**. Cero tablas de resultado.
- No engancha ningún hook de pedido** (`actionValidateOrder` , `actionOrderStatusUpdate` , override de `Order` ...).
- `check()` devuelve `false` a secas en 21 sitios distintos, **sin motivo**.
- No hay ID de regla en el pedido**. `ps_orders.total_shipping` es un número opaco.

El cálculo es 100 % efímero. Tras validar el pedido, la única huella es un importe.

7.1 El Simulador (añadido por Ecom Experts)

Para responder a "¿por qué este pedido cobró X de portes?" se ha añadido un **simulador de coste de transporte** (pestaña **Transporte** → **Simulador de coste de envío**, controlador `AdminOrderFeesShippingSimulator`). Ver [DOC_SIMULADOR.md](#).

Resumen técnico:

Pieza	Rol
<code>classes/OfShippingSimulator.php</code>	Construye un carrito temporal (INSERT directo, sin disparar hooks), ejecuta el motor real sobre él, y lo destruye.
<code>classes/OfRuleTracer.php</code>	Sondas de aislamiento : clona la regla, neutraliza todas sus restricciones menos una, y ejecuta el <code>check()</code> real del vendor. El veredicto siempre lo emite el código del vendor → cero líneas de <code>ShippingRule.php</code> modificadas.
<code>classes/OfRuleExplainer.php</code>	Traduce cada causa a "la regla exige X / el carrito tiene Y". Solo redacta, nunca decide .
<code>classes/OfSimulation.php</code>	Historial de simulaciones (5 tablas propias).

Autovalidación: `OfRuleTracer::expectedCost()` predice el coste final a partir de la traza (según el modo terminal: SUMA / SUSTITUYE / GRATIS / BASE_NATIVA / OCULTA_CARRIER) y se contrasta con el que devolvió el motor real. Si no cuadra ($> 0,01$), se marca **divergencia** en rojo en vez de presentar un desglose por regla que podría ser falso.

Para el diagnóstico manual de pedidos **pasados** (queries SQL) y el plan de instrumentación permanente, ver [DOC_DESARROLLO.md §3 y §4](#).

8. Requisitos y compatibilidad

- **Overrides OBLIGATORIOS.** Con `PS_DISABLE_OVERRIDES = 1` (Rendimiento → "Desactivar todos los overrides") el módulo **no calcula nada**. El propio controlador muestra un aviso permanente.
- Un único `specs/1.7.7.1` cubre PS $\geq 1.7.7.1$, incluidos PS 8 y PS 9. **La firma de `CartCore::getPackageShippingCost()` en PS 9 no ha sido verificada contra el core** → riesgo de fatal en el front.
- **Conflicto de override:** si otro módulo overridea `getPackageShippingCost()` o `getTotalWeight()`, la instalación falla o se instala a medias y **las reglas dejan de aplicarse en silencio**.
- **Dependencia con `orderfees`:** si el módulo hermano está activo, `orderfees_shipping` **no instala su override** y depende del de `orderfees`.
- PHP 7.4 / 8.x. Hay **2 `TypeError` latentes en PHP 8** (ver [DOC_DESARROLLO.md §5](#)) y nullable implícitos que serán fatales en PHP 9.

orderfees_shipping — Documentación de Desarrollo

Documento dirigido al desarrollador que va a **mantener, extender o depurar** el módulo. Prerrequisito: leer [DOC_TECNICO.md](#) (arquitectura, modelo de datos y motor de reglas).

1. Mapa de intervención rápida

Quiero...	Toco...
Cambiar el orden o el corte del bucle de reglas	<code>orderfees_shipping.php:159-327</code> (<code>hookActionCartGetPackageShippingCost</code>)
Añadir un tipo de condición nuevo	<code>ShippingRule::check()</code> (<code>:680-1186</code>) + tabla nueva + <code>afterAdd()</code> / <code>afterUpdate()</code> del controlador + <code>.tpl</code> de <code>conditions</code>
Añadir un tipo de cálculo nuevo	Constante de bit en <code>ShippingRule.php:112-131</code> + rama en <code>value()</code> (<code>:1188-1383</code>) + <code>tabs/actions.tpl</code>
Añadir una variable de fórmula	<code>ShippingRule::value()</code> (<code>:1263-1348</code>) + <code>\$test_data</code> en <code>AdminOrderFeesShippingController.php:274-294</code> + desplegable en <code>tabs/actions.tpl:100-113</code>
Añadir una función al parser	<code>classes/phpmathparser/expressions/HelperFunction.php</code>
Cambiar el override	<code>override/classes/Cart.php</code> Y <code>specs/1.7.7.1/override/classes/Cart.php</code> (los dos, siempre)
Instrumentar / loguear	Ver §4

2. Puntos de extensión SIN tocar el módulo

El módulo dispara estos hooks; un módulo satélite puede engancharlos:

Hook	Dónde	Qué te da
<code>actionShippingRuleCheck</code>	<code>ShippingRule.php:1171</code>	⚠ Solo se dispara si la regla YA superó TODAS las restricciones. Te dice qué reglas sí matchean, nunca por qué fallan . Devolver un valor falsy la descarta.
<code>actionShippingRuleValue</code>	<code>ShippingRule.php:1201</code>	Permite sustituir el valor calculado.
<code>actionCartGetPackageShippingCost</code>	<code>override/classes/Cart.php:23</code>	Todos los parámetros por referencia : puedes forzar <code>total</code> y <code>return</code> .
<code>actionCartGetTotalWeight</code>	<code>override/classes/Cart.php:58</code>	Ídem con <code>total_weight</code> .
<code>actionShippingRuleDuplicate</code>	Controlador	Al duplicar una regla.
<code>actionAdminOrderFeesShippingValidateBefore / After</code> , <code>...RenderForm</code> , <code>...AjaxProcess</code>	Controlador	Extender el back-office.

Limitación clave: con estos hooks puedes saber **qué reglas pasaron y qué valor produjeron**, pero **NO el motivo del rechazo**. Para el "¿por qué NO se aplicó la regla que yo esperaba?", la instrumentación de §4 Fase 2 es obligatoria.

3. Cómo averiguar por qué un pedido recibió un coste de envío determinado

3.1 El problema de raíz

El cálculo es **efímero**. El módulo no persiste nada: ni qué regla ganó, ni qué condiciones se evaluaron, ni el valor intermedio. Tras validar el pedido solo queda `ps_orders.total_shipping`, un número opaco.

Consecuencia: para pedidos **pasados** solo puedes **reconstruir** el cálculo a mano (§3.2). Para pedidos **futuros**, hay que **instrumentar** (§4).

3.2 Qué NO es reconstruible a posteriori (ni instrumentando ahora)

Dato	Por qué
<code>time_from / time_to (check() :691-692)</code>	Usan <code>date('H:i:s')</code> en el momento del cálculo del carrito , que no es necesariamente la hora del <code>date_add</code> del pedido.
<code>Currency->conversion_rate (value() :1246-1256)</code>	Se usa la tasa actual , no la histórica.
<code>Stock (outofstock , :1719)</code>	<code>StockAvailable</code> actual .
Precios / promociones	<code>getOrderTotal()</code> recalcula sobre precios actuales del producto.
La propia regla	No hay versionado, solo <code>date_add / date_upd</code> . Si el admin editó la regla después del pedido, la reconstrucción es FALSA.

Regla de oro: antes de sacar conclusiones, comprueba siempre `of_shipping_rule.date_upd` **contra** `orders.date_add`.

3.3 Queries SQL de diagnóstico

Sustituye `PREFIX_` (normalmente `ps_`). ⚠ `of_shipping_rule_carrier.id_carrier` **guarda el `id_reference`, NO el `id_carrier`**. Es el error nº 1 al depurar a mano.

```
SET @id_order := 12345;
```

1) Datos maestros del pedido y su carrito

```
SELECT o.id_order, o.reference, o.date_add, o.id_cart, o.id_customer, o.id_currency, o.id_shop,
       o.id_carrier, o.id_address_delivery, o.id_address_invoice,
       o.total_paid_tax_excl, o.total_products,
       o.total_shipping, o.total_shipping_tax_excl, o.total_shipping_tax_incl, o.carrier_tax_rate,
       c.delivery_option, c.date_upd AS cart_date_upd
FROM PREFIX_orders o
JOIN PREFIX_cart c ON c.id_cart = o.id_cart
WHERE o.id_order = @id_order;
```

2) Dirección usada (CP / ciudad / provincia / país / zona)

El módulo usa `cart.id_address_delivery` para las restricciones geográficas (`check()` :771, :795, :832, :1056, :1089, :1114) pero `PS_TAX_ADDRESS_TYPE` para los impuestos (`orderfees_shipping.php:137-145`). Saca las dos.

```
SELECT 'delivery' AS tipo, a.id_address, a.postcode, a.city, a.id_state, s.name AS state,
       a.id_country, cl.name AS country, co.id_zone, z.name AS zone
FROM PREFIX_orders o
JOIN PREFIX_address a ON a.id_address = o.id_address_delivery
JOIN PREFIX_country co ON co.id_country = a.id_country
JOIN PREFIX_country_lang cl ON cl.id_country = co.id_country AND cl.id_lang = 1
LEFT JOIN PREFIX_state s ON s.id_state = a.id_state
JOIN PREFIX_zone z ON z.id_zone = co.id_zone
WHERE o.id_order = @id_order
UNION ALL
SELECT 'invoice', a.id_address, a.postcode, a.city, a.id_state, s.name,
       a.id_country, cl.name, co.id_zone, z.name
FROM PREFIX_orders o
JOIN PREFIX_address a ON a.id_address = o.id_address_invoice
JOIN PREFIX_country co ON co.id_country = a.id_country
JOIN PREFIX_country_lang cl ON cl.id_country = co.id_country AND cl.id_lang = 1
LEFT JOIN PREFIX_state s ON s.id_state = a.id_state
JOIN PREFIX_zone z ON z.id_zone = co.id_zone
WHERE o.id_order = @id_order;
```

3) Líneas del carrito con peso, dimensiones, categoría, fabricante, proveedor

```
SELECT cp.id_product, cp.id_product_attribute, cp.quantity, pl.name,
       COALESCE(NULLIF(pa.weight,0), p.weight) AS weight_unit,
       p.width, p.height, p.depth,
       (p.width * p.height * p.depth) AS volume_unit,
       (p.width + p.height + p.depth) AS combined_unit,
       (2*p.height + 2*p.width + p.depth) AS combined_girth_unit,
       COALESCE(NULLIF(pa.weight,0), p.weight) * cp.quantity AS weight_line,
       p.id_category_default, p.id_manufacturer, p.id_supplier, p.wholesale_price,
       od.total_price_tax_excl, od.total_price_tax_incl, od.reduction_amount_tax_incl
FROM PREFIX_orders o
JOIN PREFIX_cart_product cp ON cp.id_cart = o.id_cart
JOIN PREFIX_product p ON p.id_product = cp.id_product
LEFT JOIN PREFIX_product_attribute pa ON pa.id_product_attribute = cp.id_product_attribute
LEFT JOIN PREFIX_product_lang pl ON pl.id_product = p.id_product AND pl.id_lang = 1 AND pl.id_shop = o.id_shop
LEFT JOIN PREFIX_order_detail od ON od.id_order = o.id_order
    AND od.product_id = cp.id_product AND od.product_attribute_id = cp.id_product_attribute
WHERE o.id_order = @id_order;
```

4) ¿Alguna regla de PESO (type & 1024) alteró el peso? — SEÑAL FORENSE CLAVE

ps_order_carrier.weight guarda el peso tal como lo devolvió Cart::getTotalWeight() **overrideado**. Si no coincide con la suma real de los productos, hubo una regla de peso.

```
SELECT oc.weight AS peso_registrado_en_pedido,
       (SELECT SUM(COALESCE(NULLIF(pa.weight,0), p.weight) * cp.quantity)
        FROM PREFIX_cart_product cp
        JOIN PREFIX_product p ON p.id_product = cp.id_product
        LEFT JOIN PREFIX_product_attribute pa ON pa.id_product_attribute = cp.id_product_attribute
        WHERE cp.id_cart = o.id_cart) AS peso_real_productos,
       oc.shipping_cost_tax_excl, oc.shipping_cost_tax_incl,
       oc.id_carrier, ca.id_reference, ca.name, ca.shipping_method, ca.is_free
FROM PREFIX_orders o
JOIN PREFIX_order_carrier oc ON oc.id_order = o.id_order
JOIN PREFIX_carrier ca ON ca.id_carrier = oc.id_carrier
WHERE o.id_order = @id_order;

-- Si peso_registrado != peso_real:
SELECT * FROM PREFIX_of_shipping_rule WHERE active = 1 AND (type & 1024);
```

5) Coste base del transportista (¿la regla SUSTITUYÓ o SUMÓ?)

```
SELECT d.id_carrier, d.id_zone, d.price,
       rw.delimiter1 AS w_min, rw.delimiter2 AS w_max,
       rp.delimiter1 AS p_min, rp.delimiter2 AS p_max,
       ca.shipping_method -- 1 = por peso, 2 = por precio
FROM PREFIX_orders o
JOIN PREFIX_carrier ca ON ca.id_carrier = o.id_carrier
JOIN PREFIX_delivery d ON d.id_carrier = ca.id_carrier
LEFT JOIN PREFIX_range_weight rw ON rw.id_range_weight = d.id_range_weight
LEFT JOIN PREFIX_range_price rp ON rp.id_range_price = d.id_range_price
WHERE o.id_order = @id_order
    AND d.id_zone = (SELECT co.id_zone FROM PREFIX_address a
                    JOIN PREFIX_country co ON co.id_country = a.id_country
                    WHERE a.id_address = o.id_address_delivery)
ORDER BY w_min, p_min;
```

Interpretación del resultado:

Observación	Conclusión
<code>total_shipping_tax_excl == precio_base + X</code>	Regla acumulativa (<code>BASIC_SHIPPING_MERGE</code> , type & 32)
<code>total_shipping_tax_excl == X</code> a secas	<code>BASIC_SHIPPING_RULE</code> (type & 64) — sustituyó el porte
<code>total_shipping_tax_excl == 0</code>	<code>IS_FREE_SHIPPING</code> (type & 1) o el carrier tiene <code>is_free</code>
El transportista esperado no aparecía en el checkout	<code>IS_CARRIER</code> (type & 16) lo ocultó

6) Reglas candidatas, en el ORDEN EXACTO en que las evalúa el módulo (réplica de `orderfees_shipping.php:115-126` , ampliada con el descifrado del bitmask y el aviso de modificación posterior)

```
SELECT sr.id_of_shipping_rule, sr.name, sr.priority, sr.active, sr.type,
       (sr.type & 1)    AS IS_FREE_SHIPPING,
       (sr.type & 2)    AS IS_PERCENT,
       (sr.type & 4)    AS IS_AMOUNT,
       (sr.type & 8)    AS IS_FORMULA,
       (sr.type & 16)   AS IS_CARRIER,
       (sr.type & 32)   AS BASIC_SHIPPING_MERGE,
       (sr.type & 64)   AS BASIC_SHIPPING_RULE,  -- SUSTITUYE y corta el bucle
       (sr.type & 128)  AS BASIC_SHIPPING_BASE,  -- deja el coste PS y corta
       (sr.type & 256)  AS APPLY_IF_LEAST,
       (sr.type & 512)  AS APPLY_IF_ALL,
       (sr.type & 1024) AS IS_WEIGHT,
       sr.percent, sr.amount, sr.formula, sr.currency, sr.tax_rules_group,
       sr.product, sr.quantity_per_product,
       sr.date_from, sr.date_to, sr.time_from, sr.time_to, sr.id_customer,
       sr.minimum_amount, sr.minimum_amount_restriction, sr.minimum_amount_tax,
       sr.maximum_amount, sr.maximum_amount_restriction, sr.maximum_amount_tax,
       sr.country_restriction, sr.zone_restriction, sr.state_restriction,
       sr.city_restriction, sr.zipcode_restriction, sr.carrier_restriction,
       sr.group_restriction, sr.product_restriction, sr.shop_restriction,
       sr.dimension_restriction, sr.package_restriction, sr.gender_restriction,
       sr.of_shipping_rule_restriction, sr.nb_supplier_min, sr.nb_supplier_max,
       GROUP_CONCAT(DISTINCT c.id_carrier) AS carriers_reales,
       sr.date_upd,
       IF(sr.date_upd > (SELECT date_add FROM PREFIX_orders WHERE id_order = @id_order),
          'REGLA MODIFICADA TRAS EL PEDIDO', 'ok') AS aviso
FROM PREFIX_of_shipping_rule sr
LEFT JOIN PREFIX_of_shipping_rule_carrier src ON sr.id_of_shipping_rule = src.id_of_shipping_rule
LEFT JOIN PREFIX_carrier c ON c.id_reference = src.id_carrier AND c.deleted = 0
WHERE sr.active = 1
GROUP BY sr.id_of_shipping_rule
ORDER BY sr.priority ASC;
```

7) Restricciones geográficas de cada regla, CONTRASTADAS con la dirección del pedido

```

SET @id_addr      := (SELECT id_address_delivery FROM PREFIX_orders WHERE id_order = @id_order);
SET @id_country   := (SELECT id_country FROM PREFIX_address WHERE id_address = @id_addr);
SET @id_state     := (SELECT id_state FROM PREFIX_address WHERE id_address = @id_addr);
SET @id_zone      := (SELECT id_zone FROM PREFIX_country WHERE id_country = @id_country);
SET @postcode     := (SELECT LOWER(TRIM(postcode)) FROM PREFIX_address WHERE id_address = @id_addr);
SET @city         := (SELECT LOWER(TRIM(city)) FROM PREFIX_address WHERE id_address = @id_addr);
SET @id_customer := (SELECT id_customer FROM PREFIX_orders WHERE id_order = @id_order);
SET @id_carrier   := (SELECT id_carrier FROM PREFIX_orders WHERE id_order = @id_order);
SET @ref_carrier  := (SELECT id_reference FROM PREFIX_carrier WHERE id_carrier = @id_carrier);

SELECT sr.id_of_shipping_rule AS id, sr.name, sr.priority,
  IF(sr.country_restriction=0,'-', IF(EXISTS(SELECT 1 FROM PREFIX_of_shipping_rule_country x
    WHERE x.id_of_shipping_rule=sr.id_of_shipping_rule AND x.id_country=@id_country),'PASA','FALLA')) AS f_country,
  IF(sr.zone_restriction=0,'-', IF(EXISTS(SELECT 1 FROM PREFIX_of_shipping_rule_zone x
    WHERE x.id_of_shipping_rule=sr.id_of_shipping_rule AND x.id_zone=@id_zone),'PASA','FALLA')) AS f_zone,
  IF(sr.state_restriction=0,'-', IF(EXISTS(SELECT 1 FROM PREFIX_of_shipping_rule_state x
    WHERE x.id_of_shipping_rule=sr.id_of_shipping_rule AND x.id_state=@id_state),'PASA','FALLA')) AS f_state,
  IF(sr.carrier_restriction=0,'-', IF(EXISTS(SELECT 1 FROM PREFIX_of_shipping_rule_carrier x
    WHERE x.id_of_shipping_rule=sr.id_of_shipping_rule AND x.id_carrier=@ref_carrier),'PASA','FALLA')) AS f_carrier,
  IF(sr.group_restriction=0,'-', IF(EXISTS(SELECT 1 FROM PREFIX_of_shipping_rule_group x
    JOIN PREFIX_customer_group cg ON cg.id_group = x.id_group AND cg.id_customer = @id_customer
    WHERE x.id_of_shipping_rule=sr.id_of_shipping_rule),'PASA','FALLA')) AS f_group,
  IF(sr.city_restriction=0,'-', 'REVISAR') AS f_city,
  IF(sr.zipcode_restriction=0,'-', 'REVISAR') AS f_zipcode,
  IF(sr.dimension_restriction=0,'-', 'REVISAR') AS f_dimension,
  IF(sr.product_restriction=0,'-', 'REVISAR') AS f_product,
  IF(sr.package_restriction=0,'-', 'REVISAR') AS f_package
FROM PREFIX_of_shipping_rule sr
WHERE sr.active = 1
ORDER BY sr.priority ASC;

```

8) Detalle de CP y ciudad — semántica: AND entre grupos, OR dentro del grupo

```

SELECT sr.id_of_shipping_rule, sr.name, zrg.id_zipcode_rule_group,
  zr.type AS id_country_filtro, zr.operator, zr.value, @postcode AS cp_del_pedido
FROM PREFIX_of_shipping_rule sr
JOIN PREFIX_of_shipping_rule_zipcode_rule_group zrg ON zrg.id_of_shipping_rule = sr.id_of_shipping_rule
JOIN PREFIX_of_shipping_rule_zipcode_rule zr ON zr.id_zipcode_rule_group = zrg.id_zipcode_rule_group
WHERE sr.active = 1 AND sr.zipcode_restriction = 1
ORDER BY sr.priority, zrg.id_zipcode_rule_group;

SELECT sr.id_of_shipping_rule, sr.name, crg.id_city_rule_group,
  cr.type AS id_country_filtro, cr.value, @city AS ciudad_del_pedido
FROM PREFIX_of_shipping_rule sr
JOIN PREFIX_of_shipping_rule_city_rule_group crg ON crg.id_of_shipping_rule = sr.id_of_shipping_rule
JOIN PREFIX_of_shipping_rule_city_rule cr ON cr.id_city_rule_group = crg.id_city_rule_group
WHERE sr.active = 1 AND sr.city_restriction = 1
ORDER BY sr.priority;

```

9) Reglas de bulto / peso volumétrico (réplica de `orderfees_shipping.php:267-278`; fija antes `@peso_kg` con el peso facturable)

```

SELECT sr.id_of_shipping_rule, sr.name,
  prg.id_package_rule_group, prg.unit, prg.unit_weight, prg.ratio,
  pr.range_start, pr.range_end, pr.round, pr.divider, pr.value, pr.currency, pr.tax,
  IF(pr.divider = 1, pr.value,
    ((CEIL(@peso_kg / pr.round) * pr.round) * pr.value) / pr.divider) AS precio_calculado
FROM PREFIX_of_shipping_rule sr
JOIN PREFIX_of_shipping_rule_package_rule_group prg ON prg.id_of_shipping_rule = sr.id_of_shipping_rule
JOIN PREFIX_of_shipping_rule_package_rule pr ON pr.id_package_rule_group = prg.id_package_rule_group
WHERE sr.active = 1 AND sr.package_restriction = 1
  AND CEIL(@peso_kg / pr.round) * pr.round BETWEEN pr.range_start AND pr.range_end
ORDER BY sr.priority;

```

10) Restricciones de producto

```
SELECT sr.id_of_shipping_rule, sr.name,
       prg.id_product_rule_group, prg.quantity AS qty_minima,
       pr.id_product_rule, pr.type,
       GROUP_CONCAT(prv.id_item ORDER BY prv.id_item) AS valores
FROM PREFIX_of_shipping_rule sr
JOIN PREFIX_of_shipping_rule_product_rule_group prg ON prg.id_of_shipping_rule = sr.id_of_shipping_rule
JOIN PREFIX_of_shipping_rule_product_rule pr ON pr.id_product_rule_group = prg.id_product_rule_group
LEFT JOIN PREFIX_of_shipping_rule_product_rule_value prv ON prv.id_product_rule = pr.id_product_rule
WHERE sr.active = 1 AND sr.product_restriction = 1
GROUP BY pr.id_product_rule
ORDER BY sr.priority, prg.id_product_rule_group;
```

3.4 Procedimiento manual, paso a paso

1. **Queries 1 + 2 + 3** → fija los inputs: carrito, dirección, CP/ciudad/país/zona/provincia, líneas, pesos, dimensiones, `id_carrier` y su `id_reference`.
2. **Query 6** → lista de candidatas ordenada por `priority ASC`. **Mira la columna `aviso`**: si alguna regla candidata tiene `date_upd > order.date_add`, **para aquí** — la configuración cambió desde el pedido y cualquier conclusión es especulativa.
3. **Query 7** → descarta rápido las que fallan por país / zona / provincia / carrier / grupo.
4. **Queries 8 / 9 / 10** → para las supervivientes, valida a mano CP, ciudad, dimensiones, bultos y productos, respetando **AND entre grupos, OR dentro del grupo**.
5. **Recorre las supervivientes en orden de prioridad** aplicando la lógica de corte de `orderfees_shipping.php:219-326`: la primera con `type & 1` (FREE), `type & 64` (RULE) o `type & 128` (BASE) **corta el bucle**; el resto **acumulan**.
6. **Queries 4 + 5** → contrasta el total con `total_shipping_tax_excl` y con el rango base del carrier: confirma si sustituyó o sumó, y si hubo alteración de peso.
7. Si el importe **no cuadra**: sospecha de `time_from / time_to`, de la tasa de cambio o de precios cambiados. **Esas condiciones no son reconstruibles** → **hay que instrumentar (§4)**.

4. Plan de instrumentación (para responder la pregunta EN AUTOMÁTICO)

Cumplimiento de la norma de seguridad del proyecto: nada de esto es un scan sobre histórico ni dispara acciones hacia terceros. Solo escribe en BD/fichero **local** durante el cálculo del carrito del cliente actual. **NO hacer backfill del histórico:** los datos necesarios (hora del cálculo, tasa de cambio, stock, precios del momento) ya no existen; cualquier "reconstrucción masiva" produciría datos falsos. **Instrumentar desde hoy hacia adelante** y usar §3.3 para los pedidos pasados.

Fase 0 — Flag de depuración

Añadir en `orderfees_shipping.php:63` (array `$configurations`):

```
array('name' => 'MS_ORDERFEES_SHIPPING_DEBUG', 'label' => 'Debug: registrar decision de reglas', 'default' => '0'),
```

El flag es obligatorio: el hook se ejecuta O(paquetes × transportistas × reglas) veces por página. Un log siempre activo revienta el disco.

Fase 1 — Logger

`classes/OfsLogger.php` + `logs/index.php` + `logs/.htaccess` (Deny from all). Rutas **dentro de** `_PS_MODULE_DIR_ . 'orderfees_shipping/logs/'`, **NUNCA** `/var/logs` ni `/tmp`.

```

class OfsLogger
{
    protected static $buffer = array();

    public static function enabled()
    {
        return (bool) Configuration::get('MS_ORDERFEES_SHIPPING_DEBUG');
    }

    public static function log($event, array $data = array())
    {
        if (!self::enabled()) {
            return;
        }
        self::$buffer[] = array('t' => microtime(true), 'event' => $event) + $data;
        @file_put_contents(
            _PS_MODULE_DIR_ . 'orderfees_shipping/logs/ofs-' . date('Y-m-d') . '.log',
            date('H:i:s') . ' [' . $event . ']' . ' ' . json_encode($data) . PHP_EOL,
            FILE_APPEND
        );
    }

    public static function flush() { $b = self::$buffer; self::$buffer = array(); return $b; }
    public static function getBuffer() { return self::$buffer; }
}

```

Fase 2 — Motivo de rechazo en `check()` — EL PUNTO CRÍTICO

Es lo que responde el 90 % de los "*¿por qué no me aplicó la regla?*".

En `classes/ShippingRule.php`, antes de `check()` (línea 680):

```

public $reject_reason = null;

protected function reject($reason, array $ctx = array())
{
    $this->reject_reason = $reason;
    OfsLogger::log('rule_reject', array(
        'id_rule' => (int) $this->id,
        'name' => $this->name,
        'priority' => (int) $this->priority,
        'reason' => $reason,
    ) + $ctx);

    return false;
}

```

Y sustituir **cada uno de los 21** `return false;` de `check()` (rango 680-1186) por `return $this->reject('<motivo>', [...]);`:

Línea aprox.	Motivo	Contexto útil
694	main	active / date_from / date_to / time_from / time_to / id_customer
729	minimum_amount	cart_total, minimum, restriction
765	maximum_amount	cart_total, maximum
772	city_no_address	—
789	city_cityselect	city
818	city_rule_group	id_city_rule_group, city
826	zipcode_no_address	—
890	zipcode_rule_group	id_zipcode_rule_group, postcode
1005	dimension_rule_group	id_dimension_rule_group, base, dims calculadas
1012	product_restriction	—
1031	group_restriction	—
1063	country_restriction	id_country
1097	zone_restriction	id_zone
1104	state_no_address	—
1120	state_restriction	id_state
1134	shop_restriction	id_shop
1153	gender_restriction	—
1167	nb_suppliers	nb, min, max
1180	hook_actionShippingRuleCheck	módulo que la vetó
1185	return true; → antes, OfsLogger::log('rule_pass', [...])	

⚠ **Las líneas son aproximadas.** Antes de aplicar esta fase, releer ShippingRule.php:680-1186 y confirmar cada return false; uno a uno (son 21 y algunos están dentro de bucles anidados).

Además: - ShippingRule.php:1195-1199 (value()): log de **cache HIT** con \$cache_id (delata el bug de caché sin id_carrier, \$5 nº 6). - ShippingRule.php:1380 (antes de Cache::store): OfsLogger::log('rule_value', ['id_rule', 'value', 'type', 'percent', 'amount', 'formula', 'use_tax']). - ShippingRule.php:1922 (parse()): loguear formula + data + resultado (delata las fórmulas que fallan **en silencio**). - ShippingRule.php:1866 (factory()): punto natural para inyectar el logger.

Fase 3 — Trazar la resolución del bucle

orderfees_shipping.php, hookActionCartGetPackageShippingCost() :

Línea	Qué añadir
107-109	"SKIP: disable_calculation activo"
113	<code>OfsLogger::log('calc_start', ['id_cart', 'id_carrier', 'use_tax']);</code>
127	<code>OfsLogger::log('candidates', ['ids' => array_column(\$items, 'id_of_shipping_rule')]);</code>
156	<code>OfsLogger::log('context', ['id_address', 'id_carrier', 'id_reference']);</code>
159	Inicio del foreach: <code>id_regla</code> , <code>priority</code> , <code>type</code> descodificado
162-164	"SKIP: regla desactivada por combinación"
170	Resultado de <code>check()</code> ← el más importante
185	<code>OfsLogger::log('decision', ['action' => 'HIDE_CARRIER', 'total' => false]);</code>
214	<code>OfsLogger::log('rule_disabled_by_combination', ['by' => X, 'disabled' => Y]);</code>
224	<code>OfsLogger::log('weight_rule_queued', ['id_rule', 'formula']);</code>
232	<code>OfsLogger::log('decision', ['action' => 'FREE_SHIPPING', 'total' => 0]);</code>
304	<code>weight</code> , <code>volume</code> , <code>volumetric_weight</code> , <code>unit</code> , <code>ratio</code> , <code>id_package_rule</code> , <code>price</code> , <code>total</code>
309	Valor devuelto por <code>value()</code> + <code>total</code> redondeado
317	<code>OfsLogger::log('decision', ['action' => 'REPLACE', 'total' => \$total]);</code>
322	<code>OfsLogger::log('decision', ['action' => 'KEEP_PS_BASE']);</code>
326	<code>OfsLogger::log('decision', ['action' => 'ADD', 'delta' => \$total, 'running_total' => \$params['total']]);</code>

hookActionCartGetTotalWeight() :

Línea	Qué añadir
332-334	"SKIP: weight_rules vacío" ← delata el bug del estado estático (§5, riesgo ALTO)
343-364	<code>id_regla</code> , <code>selected_products</code> , <code>all_products</code> , <code>data['quantity']</code> , <code>data['weight']</code>
369	<code>OfsLogger::log('weight_rule_applied', ['id_rule', 'formula', 'data', 'result']);</code>

Override `override/classes/Cart.php` (replicar **idéntico** en `specs/1.7.7.1/override/classes/Cart.php`, offsets +1/+2):

Punto	Línea (base / spec)	Qué loguear
A. Entrada	23 / 24	<code>id_cart</code> , <code>id_carrier</code> , <code>use_tax</code> , <code>id_zone</code> , <code>count(\$product_list)</code> , <code>debug_backtrace(..., 3)</code> para saber quién pidió el precio (<code>getDeliveryOptionList</code> vs <code>getOrderTotal</code>)
B. Decisión terminal	34-36 / 36-38	<code>return = true</code> , <code>total</code> crudo, valor final. Si <code>\$total === false</code> → "TRANSPORTISTA DESACTIVADO por regla"
C. Precio nativo PS	45 / 48	<code>\$shipping_cost</code>
D. MERGE	46-48 / 49-51	<code>nativo</code> , <code>extra_reglas</code> , <code>final</code>
E. Sin transportista	50 / 53	"PS nativo devolvió false"
F/G. Peso	58 y 65-69 / 61 y 68-72	<code>peso_nativo</code> , <code>peso_reglas</code> , <code>peso_final</code>

Fase 4 — Persistencia por pedido (resuelve el problema de raíz)

1) Tabla nueva — crear vía función `addnewfeatures()` con `try{}` invocada al entrar en la configuración (norma del proyecto), **NO con fichero** `upgrade` :

```
CREATE TABLE IF NOT EXISTS `PREFIX_of_shipping_rule_applied` (
  `id_applied`          INT(10) UNSIGNED NOT NULL AUTO_INCREMENT,
  `id_order`            INT(10) UNSIGNED NOT NULL DEFAULT 0,
  `id_cart`             INT(10) UNSIGNED NOT NULL,
  `id_carrier`          INT(10) UNSIGNED NOT NULL DEFAULT 0,
  `id_of_shipping_rule` INT(10) UNSIGNED NOT NULL DEFAULT 0,
  `rule_name`           VARCHAR(255) NOT NULL DEFAULT '',
  `rule_priority`       INT(10) NOT NULL DEFAULT 0,
  `rule_type`           INT(10) NOT NULL DEFAULT 0,
  `action`              VARCHAR(32) NOT NULL DEFAULT '', -- ADD|REPLACE|FREE_SHIPPING|KEEP_PS_BASE|HIDE_CARRIER|REJECT
  `reject_reason`       VARCHAR(64) NULL,
  `value`               DECIMAL(17,6) NOT NULL DEFAULT 0,
  `running_total`       DECIMAL(17,6) NOT NULL DEFAULT 0,
  `trace`               TEXT NULL, -- JSON completo del buffer
  `date_add`            DATETIME NOT NULL,
  PRIMARY KEY (`id_applied`),
  KEY `id_order` (`id_order`),
  KEY `id_cart` (`id_cart`)
) ENGINE=_MYSQL_ENGINE_ DEFAULT CHARSET=UTF8;
```

2) Snapshot por carrito — al final de `hookActionCartGetPackageShippingCost()` (y en cada `return`), volcar `OfsLogger::getBuffer()` con `id_order = 0`, haciendo un `DELETE` previo por `(id_cart, id_carrier)` para no duplicar en cada recálculo del checkout.

3) Vincular al pedido — registrar el hook nuevo en `orderfees_shipping.php:97-103`:

```
&& $this->registerHook('actionValidateOrder')
```

```
public function hookActionValidateOrder($params)
{
    $order = $params['order'];
    Db::getInstance()->update(
        'of_shipping_rule_applied',
        array('id_order' => (int) $order->id),
        'id_cart = ' . (int) $order->id_cart . ' AND id_order = 0'
    );
}
```

4) Visualización — hook `displayAdminOrder` que muestre las filas de `of_shipping_rule_applied WHERE id_order = X`. **Ahí se responde la pregunta original en un clic**, sin SQL a mano.

Alternativa sin tocar el módulo (menos potente, cero riesgo)

Módulo satélite que enganche `actionShippingRuleCheck`, `actionShippingRuleValue`, `actionCartGetPackageShippingCost` y `actionValidateOrder`. Da **qué reglas pasaron y qué valor produjeron**, pero **NO el motivo del rechazo** (ese hook solo se dispara si la regla ya superó todas las restricciones). Para el *"¿por qué NO se aplicó la regla que esperaba?"*, **la Fase 2 es obligatoria**.

5. Bugs confirmados

#	Fichero:línea	Problema	Severidad
1	ShippingRule.php:1796-1859	<code>getShippingCostByProduct()</code> está ROTO : llama <code>value(true)</code> / <code>value(false)</code> pasando un bool cuando <code>value(\$params)</code> espera un array → TypeError fatal en PHP 8 . Solo lo usa <code>hookActionCartSummary</code> , que no está registrado — por eso no ha explotado.	Latente
2	ShippingRule.php:1460	<code>implode(array_map('intval', \$shops), ',')</code> — argumentos invertidos → TypeError en PHP 8.0+ al abrir el formulario con multitienda.	Alta
3	ShippingRule.php:760	Usa <code>\$this->minimum_amount_tax</code> dentro del bloque de maximum amount (copy-paste).	Media
4	ShippingRule.php:1158	La restricción de proveedores solo se evalúa si <code>nb_supplier_max</code> es truthy → definir solo el mínimo no hace nada .	Media
5	ShippingRule.php:1770-1774	Caché estática de <code>getProducts()</code> indexada solo por <code>\$this->id</code> , sin <code>id_cart</code> → contaminación entre carritos en la misma petición.	Media
6	ShippingRule.php:1195	Clave de caché de <code>value()</code> sin <code>id_carrier</code> ni <code>\$params['total']</code> → una fórmula que use <code>shipping</code> devuelve el mismo valor para todos los transportistas en esa petición.	Alta
7	ShippingRule.php:281-393	<code>copyConditions()</code> NO copia dimension / city / zipcode / package rule groups. (El botón "Duplicar" del listado usa <code>duplicateTables()</code> (<code>AdminOrderFeesShippingController.php:450</code>), que sí los copia → el bug es latente, no visible al usuario.)	Latente
8	ShippingRule.php:158-279	<code>delete()</code> no purga <code>of_shipping_rule_city</code> → filas huérfanas. Usa <code>&=</code> (bitwise) en vez de <code>&&</code> .	Baja
9	ShippingRule.php:873-885	Comparación de CP como STRINGS (<code>></code> , <code><</code>) → CP con ceros a la izquierda comparan mal (<code>'09999' < '1'</code> es TRUE).	Media
10	ShippingRule.php:1210 vs :1217	<code>IS_PERCENT</code> sobre toda la orden usa SIN IVA hardcoded, pero sobre un producto usa <code>total_wt</code> (CON IVA) → incoherencia fiscal .	Media
11	ShippingRule.php:960	Si ningún producto tiene dimensiones , la restricción de dimensión se ignora en vez de fallar.	Media
12	orderfees_shipping.php:299	<code>\$total</code> se SOBREScribe (no acumula) en el bucle de <code>package_rule_groups</code> → con varios grupos, solo prevalece el último.	Media
13	phpmathparser/expressions/Unary.php	<code>protected \$precedence = 7;</code> — typo (falta la <code>e</code>) → la precedencia efectiva del menos unario es 0 , la más baja.	Media
14	phpmathparser/Math.php:152	El <code>';</code> no está en la lista de tokens tras los que un <code>-</code> se hace unario → MIN(10; -5) produce un resultado silenciosamente INCORRECTO .	Alta
15	ShippingRule.php:133 + :1922	Singleton de <code>Math</code> sin reset : <code>registerVariable()</code> acumula . Las 19 variables de una regla de coste siguen registradas cuando después se evalúa una regla de peso (que solo registra 2) → una fórmula de peso puede referirse a <code>total</code> y evaluar un valor obsoleto .	Alta
16	phpmathparser/expressions/MathFunction.php:17	<code>PI</code> , <code>ATAN2</code> , <code>DEG2GRAD</code> declaradas pero fatalizan si se usan. <code>ROUND()</code> solo acepta 1 argumento.	Baja
17	AdminOrderFeesShippingController.php:970-976	<code>if (Tools::isSubmit('newCityRule'))</code> duplicado → el segundo bloque (que pretendía manejar <code>zipcode_rule_*</code>) es código muerto . No rompe nada.	Baja
18	views/js/admin.js (addPackageRule)	<code>parseFloat(...)</code> == '' nunca se cumple (<code>parseFloat</code> devuelve <code>NaN</code>) → se puede añadir un rango vacío que llega como <code>NaN</code> al AJAX.	Baja
19	views/js/admin.js (addProductRule)	El contador se incrementa antes de validar el tipo → huecos en los IDs.	Baja
20	views/js/admin.js	<code>jscroll</code> + <code>typeWatch</code> apuntan a <code>#shipping_rule_select_1/_2</code> pero <code>conditions.tpl</code> genera <code>of_shipping_rule_select_1/_2</code> → la búsqueda	Baja

#	Fichero:línea	Problema	Severidad
		paginada de reglas combinables NUNCA se activa. Usa además una global <code>baseHref</code> no definida.	
21	<code>ShippingRule.php:1521</code>	<code>MS_ORDERFEES_SHIPPING_CITYSELECT</code> leída pero nunca declarada ni inicializada.	Baja

6. Riesgos de arquitectura y operación

Riesgo	Severidad	Detalle
Conflicto de override	ALTO	PrestaShop fusiona overrides en un único <code>override/classes/Cart.php</code> . Si otro módulo declara <code>getPackageShippingCost()</code> o <code>getTotalWeight()</code> → <code>Module::addOverride()</code> lanza <i>"The method X in the class Cart is already overridden"</i> → la instalación falla , o (peor) se instala a medias y las reglas dejan de aplicarse en silencio . Módulos típicamente incompatibles: transportistas avanzados, matrices de envío, "free shipping" de terceros.
Dependencia oculta con orderfees	ALTO	<code>orderfees_shipping.php:83-90</code> : si <code>orderfees</code> está activo, este módulo NO instala su override de Cart . Si <code>orderfees</code> es de versión antigua (sin <code>actionCartGetTotalWeight()</code>), las reglas de peso dejan de funcionar sin ningún error visible . Si se desinstala <code>orderfees</code> , el override desaparece y <code>orderfees_shipping</code> deja de calcular nada .
<code>self::\$weight_rules</code> es estática y depende del orden de llamada	ALTO	Se resetea en <code>:111</code> y se consume en <code>:332</code> . <code>getTotalWeight()</code> solo aplica reglas de peso si antes, en la misma petición, se ejecutó <code>getPackageShippingCost()</code> . Cualquier consumidor de <code>Cart::getTotalWeight()</code> fuera de un cálculo de envío (filtrado de transportistas por rango de peso, BO, webservice, módulos de terceros) obtiene el peso SIN las reglas → la disponibilidad del transportista puede no coincidir con el precio mostrado .
Rendimiento	ALTO	El override llama al hook ANTES de <code>parent::</code> , así que la caché interna de <code>CartCore::getPackageShippingCost()</code> NO protege el hook . La SELECT maestra y el <code>check()</code> de todas las reglas se ejecutan en cada llamada, y <code>getDeliveryOptionList()</code> llama a <code>getPackageShippingCost()</code> una vez por paquete × transportista . Coste O(paquetes × carriers × reglas) por página. Agravado por la falta total de índices en las 8 tablas <code>*_rule_group</code> / <code>*_rule</code> sobre sus columnas de JOIN → full scan en cada refresco del carrito.
Uninstall DESTRUCTIVO	ALTO	<code>uninstall.sql</code> hace <code>DROP TABLE</code> de las 21 tablas sin backup ni confirmación . Desinstalar el módulo destruye irreversiblemente todas las reglas de portes .
<code>hookActionDiscoverModules</code> = superficie de RCE + telemetría	ALTO (seguridad)	<code>MotionSeedModule.php:736-778</code> : hace <code>file_get_contents('https://www.motionseed.com/modules-discover')</code> enviando en la cabecera <code>Referer</code> la URL de la tienda, el nombre y versión del módulo y la versión de PS. Después renderiza el XML descargado como plantilla Smarty → si el endpoint remoto se ve comprometido, es ejecución remota de código en la tienda . Se ejecuta en cada <code>configureMenu()</code> y cada 24 h.
Redondeo a 2 decimales hardcodedo	MEDIO	<code>Tools::ps_round((float) \$total, 2)</code> en <code>override/classes/Cart.php:35</code> y <code>:47</code> . Ignora <code>Currency::\$decimals</code> . Divisas de 0 decimales (JPY, KRW) dan importes incoherentes.
<code>keepOrderPrices</code> ignorado	MEDIO	El override lo pasa al hook pero <code>hookActionCartGetPackageShippingCost()</code> nunca lo lee → al recalcular el envío de un pedido existente con "mantener precios del pedido", las reglas se re-evalúan con precios actuales .
PHP 8.4+ / PHP 9	MEDIO	<code>Country \$default_country = null</code> es un nullable implícito → Deprecated en 8.4, fatal en 9 . Haría falta un <code>specs/</code> con <code>?Country \$default_country = null</code> . También <code>preg_split(\$string, null, ...)</code> en <code>Math.php</code> .
PS 9	MEDIO	Solo existe <code>specs/1.7.7.1</code> . Si PS 9 cambia la firma de <code>CartCore::getPackageShippingCost()</code> , el override rompe el front entero. Verificar contra el core de la versión objetivo .
<code>Tab::getIdFromClassName()</code>	MEDIO	<code>MotionSeedModule.php:174</code> y <code>:180</code> → lanza <code>NonUniqueResult</code> en PS9 con tabs duplicadas.
<code>Hook::exec()</code> con referencias en Symfony	MEDIO	Funciona con <code>Hook::coreCallHook()</code> (legacy), pero no con el <code>HookDispatcher</code> de Symfony del BO. Cualquier cálculo de envío disparado desde el BO Symfony (crear pedido desde back-office) puede no recibir las modificaciones por referencia .
Upgrades NO idempotentes	MEDIO	Todos los <code>upgrade.sql</code> usan <code>ADD COLUMN</code> sin comprobar <code>INFORMATION_SCHEMA</code> . upgrade-1.10.6.php es especialmente peligroso : ejecutarlo dos veces desplazaría los enumerados otra vez, corrompiendo silenciosamente todas las condiciones de importe .
<code>MS_ORDERFEES_SHIPPING_TYPES</code> obsoleta	MEDIO	<code>upgrade-1.8.26.php</code> la sobrescribe incondicionalmente y su valor nunca se actualizó para incluir <code>city</code> , <code>dimension</code> , <code>zipcode</code> , <code>package</code> ni <code>gender</code> . Si en la tienda vale <code>country</code>, <code>zone</code>, <code>state</code>, <code>carrier</code>, <code>group</code>, <code>of_shipping_rule</code>, <code>shop</code>, esas pestañas quedarían OCULTAS en el formulario aunque el motor las soporte .

Riesgo	Severidad	Detalle
CHARSET=UTF8 (no utf8mb4)	BAJO	<code>of_shipping_rule_city.name</code> no admite caracteres de 4 bytes. En PS 8/9 con BD <code>utf8mb4</code> , los JOINS textuales pueden dar <code>Illegal mix of collations</code> .
Traducciones legacy	BAJO	Sistema antiguo (<code>{1 s='...' mod='orderfees_shipping'} + \$_MODULE[md5()]</code>) en <code>translations/<iso>.php</code> , no <code>.xlf</code> . Los 4 mensajes de error del parser no están traducidos en ningún idioma → siempre salen en inglés.
JS ofuscado	BAJO	<code>views/js/admin.js</code> está ofuscado con javascript-obfuscator. Dificulta cualquier mantenimiento del BO.

7. Checklist de verificación en la tienda del cliente

Antes de diagnosticar nada, confirmar:

```
-- 1) ¿Están las pestañas de condiciones ocultas por configuración obsoleta?
SELECT value FROM PREFIX_configuration WHERE name = 'MS_ORDERFEES_SHIPPING_TYPERES';
-- Si NO incluye city/dimension/zipcode/package/gender → pestañas OCULTAS.

-- 2) ¿Están los overrides activos? (si es 1, el módulo NO calcula nada)
SELECT value FROM PREFIX_configuration WHERE name = 'PS_DISABLE_OVERRIDES';

-- 3) ¿Hay fórmulas activas? (determina si el bug de caché #6 te afecta)
SELECT id_of_shipping_rule, name, type, formula
FROM PREFIX_of_shipping_rule WHERE active = 1 AND (type & 8);

-- 4) ¿Hay reglas de peso? (determina si el bug #15 y el riesgo de $weight_rules te afectan)
SELECT id_of_shipping_rule, name, formula
FROM PREFIX_of_shipping_rule WHERE active = 1 AND (type & 1024);
```

Y a nivel de sistema:

- ¿Está instalado y activo el módulo hermano `orderfees`? (cambia radicalmente el diagnóstico: el override de Cart no sería de este módulo).
- ¿Qué **otros módulos overridean** `Cart`? Revisar el `override/classes/Cart.php` desplegado y `var/cache/*/class_index.php`.
- **Versión de PHP y de PrestaShop**: determina si los bugs #1, #2 (TypeError en PHP 8) están activos hoy.
- ¿Está instalado el módulo `cityselect`? (dependencia blanda del modo A de la restricción de ciudad).
- ¿Se han añadido **índices** a mano en las tablas hijas? `SHOW INDEX FROM PREFIX_of_shipping_rule_product_rule_group;`

8. Notas de mantenimiento

- **Al tocar el override, tocar SIEMPRE los DOS ficheros** (`override/classes/Cart.php` y `specs/1.7.7.1/override/classes/Cart.php`). Solo cambia la firma (`$keepOrderPrices`).
- `afterUpdate($rule)` (`AdminOrderFeesShippingController.php:495-587`) **BORRA TODO antes de reinsertar**: DELETE por `id_of_shipping_rule` en 9 tablas planas + los 5 árboles de rule groups. Editar y guardar una regla **reconstruye todas sus condiciones desde el POST**. Si el navegador no envía un bloque, **esa condición desaparece**.
- **La sincronización de combinaciones es bidireccional**: al activar `of_shipping_rule_restriction` en una regla, `afterAdd()` **modifica automáticamente las OTRAS reglas** (les activa el flag y les propaga las combinaciones). Comportamiento sorprendente pero intencionado.
- **Fórmulas: la validación al guardar es la única red de seguridad**. En runtime, un error de fórmula se ignora en silencio (`value()` :1345) y el recargo queda a 0.
- **No usar** `getShippingCostByProduct()` (roto, ver §5 nº 1) ni `copyConditions()` (incompleto, nº 7).

Simulador de coste de transporte

Añadido a `orderfees_shipping` por **Ecom Experts** (2026). Responde a la pregunta "¿por qué este pedido cobró X de gastos de envío?".

1. Dónde está

Back-office → Transporte → "Simulador de coste de envío"

La pestaña, las 5 tablas y las 7 claves de configuración las crea la función `addnewfeatures()` de `orderfees_shipping.php`, que se ejecuta sola:

- al **instalar** el módulo,
- al pulsar "**Configurar**",
- y al **entrar en el simulador** (self-heal, por si nunca pulsaste "Configurar").

Es idempotente: si ya está todo, no hace nada. **No hay que reinstalar el módulo.**

⚠ Requisito: el simulador **aborta** si el override de `cart` no está activo (Parámetros avanzados → Rendimiento → "Desactivar todos los overrides" = **No**). Sin él, el módulo no aplica ninguna regla y el simulador te mostraría el coste nativo haciéndote creer que tus reglas no funcionan. Prefiere fallar con un error explícito.

2. Los 3 modos de entrada

a) Un carrito que monto a mano

Escribes país, provincia, código postal, ciudad y grupo de cliente, y añades productos con autocompletado (busca por nombre, referencia o ID; incluye combinaciones).

b) La dirección de un cliente real

Buscas el cliente (por ID, email, nombre o apellidos) y eliges una de sus direcciones.

c) Un pedido existente ← el caso "cobró de más / de menos"

Metes el **ID del pedido** y pulsas "**Cargar pedido**". El simulador reconstruye el carrito (líneas del `ps_cart` del pedido, con fallback a `order_detail`), la dirección de entrega, el cliente y el transportista, y te muestra:

- Los **portes que se cobraron realmente** (`total_shipping_tax_excl` / `_incl` del pedido).
- Los **portes que las reglas calculan HOY**.
- Y la **traza** que explica de dónde sale el importe de hoy.

⚠ Dos avisos que el simulador te dará en rojo, y que hay que tomarse en serio:

1. Si **alguna regla activa se modificó después** del pedido (`of_shipping_rule.date_upd > orders.date_add`), la simulación **no reproduce** lo que pasó entonces. Una diferencia no demuestra que haya un fallo: puede ser simplemente ese cambio.
2. La simulación usa **siempre** la hora del servidor, el tipo de cambio, el stock y los precios **actuales**. Las reglas con franja horaria (`time_from` / `time_to`), con conversión de divisa o que dependen de promociones **no son reproducibles retroactivamente**. Esto es una limitación del diseño del módulo original, no del simulador.

3. Qué te devuelve

Por **cada transportista** (o solo el que elijas):

Bloque	Contenido
Importes	Coste nativo de PrestaShop (sin reglas) frente al coste con las reglas aplicadas , con y sin IVA. La diferencia entre ambos es exactamente lo que hacen tus reglas.
Peso	Peso real de los productos frente al peso que PrestaShop acaba usando tras las reglas de tipo <code>weight</code> . Si difieren, se marca ALTERADO .
Traza	Regla por regla, en orden de prioridad .

Y además lista los transportistas que **existen para esa zona pero que una regla ha ocultado** del checkout (`Show only selected carriers`) — la respuesta a "*¿por qué no me sale el transportista X?*".

La traza, columna a columna

Columna	Qué dice
Prio	Prioridad. Se evalúan de menor a mayor.
Regla	ID, nombre y el bitmask <code>type</code> descompuesto en etiquetas legibles.
¿Aplica?	SÍ / NO . Es el veredicto del <code>check()</code> real del módulo, no una opinión.
Decisión	<code>SUMA</code> · <code>SUSTITUYE</code> · <code>GRATIS</code> · <code>BASE_NATIVA</code> · <code>OCULTA_CARRIER</code> · <code>ALTERA_PESO</code> · <code>SIN_EFECTO</code> · <code>OTRO_CARRIER</code> · <code>NO_APLICA</code>
Importe	Lo que esa regla aportó.
Por qué	Si aplicó: se cumplen todas sus condiciones. Si NO aplicó: qué condición concreta falló , con "la regla exige X / el carrito tiene Y".

Cuando una regla **corta el bucle** (envío gratis, "usar solo el coste de esta regla" o "usar solo el coste básico"), aparece una fila azul avisando de que **ninguna regla por debajo llegó a evaluarse**. Ese es el origen de la mayoría de los "mi regla no se aplica".

4. Cómo sabe "por qué NO aplicó" — sondas de aislamiento

Este es el punto técnico que hace que el simulador sea fiable.

`ShippingRule::check()` devuelve `false` a secas en **21 sitios distintos** y no dice el motivo. Había dos formas obvias de resolverlo, y las dos son malas:

- **Modificar los 21 `return false`** del vendor → se pierde en cada actualización del módulo.
- **Reimplementar las condiciones** en una clase paralela → se desviaría del vendor en silencio, y podría decirte "no aplicó por el CP" cuando en realidad fue por otra cosa.

Lo que hace el simulador es distinto:

1. Ejecuta el `check()` **real** de la regla. Ese es el **oráculo**: nunca se cuestiona.
2. Si dio `false`, para **cada** restricción activa: **clona** la regla, **neutraliza todas las demás** (flags a 0, fechas y horas a null, cliente a 0...), deja **solo esa**, y vuelve a ejecutar el `check()` **real**. Si con solo esa restricción sigue fallando, **esa restricción es la causa**.

El veredicto siempre lo emite el código del vendor, con sus SQL, sus rarezas y sus bugs. **No se ha tocado ni una línea de `ShippingRule.php`**. Si mañana motionSeed cambia la lógica, el simulador sigue acertando.

Ventaja añadida: el motor real **cortocircuita** en la primera condición que falla; el simulador te da **todas** las causas.

Casos especiales que el simulador maneja:

- **Veto de un módulo de terceros** (hook `actionShippingRuleCheck`): se comprueba **el primero**. Si un módulo veta la regla, *todas* las sondas fallarían y les echaríamos la culpa a las restricciones. Detectándolo antes, se reporta el veto y no se sondea nada más.
- **Causas dependientes**: si la condición de producto falla, las de dimensiones y las de importe-sobre-productos se calculan sobre 0 productos. Se marcan como "*consecuencia de la condición de producto*", no como causas independientes.
- **Causa no identificada**: si ninguna sonda encuentra la causa, se marca en **rojo** como "*posible cambio en `ShippingRule.php`*" en vez de inventar una explicación.

Autovalidación

El simulador **predice** el coste final a partir de su propia traza (según el modo terminal: suma acumulativa, sustitución, gratis, base nativa u ocultación) y lo **contrasta** con el que devolvió el motor real. Si no cuadra por más de 0,01, muestra un banner rojo de **divergencia**: el coste final

sigue siendo correcto, pero te avisa de que el **desglose por regla** es solo orientativo. Prefiere admitir que no lo sabe antes que enseñarte números falsos.

5. Seguridad: el carrito temporal

El simulador necesita un carrito **persistido** en base de datos (`CartCore::getProducts()` y `getOrderTotal()` devuelven vacío si el carrito no tiene `id_cart` , y todas las consultas de `ShippingRule` filtran por `cart_product.id_cart`). Eso obliga a crear uno de verdad, con todo el riesgo que implica. Las protecciones:

Riesgo	Protección
Un módulo de carrito abandonado le escribe al cliente ("tu carrito te espera") con productos que nunca añadió.	Modo seguro forzado. El carrito y sus líneas se insertan con <code>Db::insert()</code> directo, que no dispara ningún hook (<code>DbCore</code> nunca llama a <code>Hook::exec()</code>). El modo no seguro (que usaría <code>Cart::add()</code> → <code>actionCartSave</code>) está bloqueado por la constante <code>OfShippingSimulator::ALLOW_UNSAFE_MODE = false</code> , aunque alguien ponga la configuración a 0.
El carrito temporal cuenta como carrito abandonado.	Se destruye en <code>finally</code> y en <code>register_shutdown_function()</code> (sobrevive a un fatal error o a un timeout). Vive milisegundos.
Un fatal error deja un carrito huérfano.	Se registra en <code>of_shipping_sim_cart</code> antes de nada más. <code>sweepOrphans()</code> lo limpia (ventana de 1 hora). Solo borra ids que el propio simulador registró — jamás hace un barrido heurístico sobre <code>ps_cart</code> .
El cliente técnico del simulador acaba siendo un cliente real.	Se crea con <code>deleted = 1</code> , <code>newsletter = 0</code> , <code>optin = 0</code> y email <code>simulator@orderfees.invalid</code> . Toda operación que lo modifique exige que el email coincida ; si la clave de configuración apuntase por error a un cliente real, se ignora y se regenera — el cliente real no se toca nunca .
El contexto de tienda queda corrupto.	Se guarda y se restaura (<code>Shop::setContext</code> con su id original).

No se toca ningún dato histórico. No se dispara ninguna acción hacia terceros. El simulador solo lee pedidos, crea un carrito nuevo y lo borra.

6. Configuración

Clave	Por defecto	Para qué
<code>MS_ORDERFEES_SHIPPING_FEATURES</code>	<code>0 → 1</code>	Versión de migración aplicada. Idempotencia.
<code>MS_ORDERFEES_SHIPPING_SIM_SAFE_MODE</code>	<code>1</code>	Modo seguro. Hoy es irrelevante: <code>ALLOW_UNSAFE_MODE = false</code> fuerza el modo seguro pase lo que pase.
<code>MS_ORDERFEES_SHIPPING_SIM_HISTORY</code>	<code>1</code>	Guardar el historial de simulaciones.
<code>MS_ORDERFEES_SHIPPING_SIM_HISTORY_DAYS</code>	<code>30</code>	Purga automática del historial.
<code>MS_ORDERFEES_SHIPPING_SIM_MAX_PROBES</code>	<code>300</code>	Tope de sondas por simulación (evita timeouts con muchas reglas). Si se alcanza, se avisa de que pueden faltar causas.
<code>MS_ORDERFEES_SHIPPING_SIM_RULEHASH</code>	—	<code>md5</code> de <code>ShippingRule.php</code> . Si cambia, avisa de que el módulo se ha actualizado.
<code>MS_ORDERFEES_SHIPPING_SIM_ID_CUSTOMER</code>	<code>0</code>	Cliente técnico del modo "dirección manual".

7. Ficheros

Nuevos:

```

classes/OfShippingSimulator.php      Carrito temporal + ejecución del motor real
classes/OfRuleTracer.php             Sondas de aislamiento + réplica del bucle de decisión
classes/OfRuleExplainer.php          Redacción de las causas (nunca decide)
classes/OfSimulation.php             ObjectModel del historial
controllers/admin/AdminOrderFeesShippingSimulatorController.php
config/sql/simulator_install.sql     5 tablas
views/templates/admin/orderfees_shipping/simulator.tpl
views/templates/admin/orderfees_shipping/simulator_result.tpl
views/js/simulator.js
views/css/simulator.css

```

Modificados:

Fichero	Cambio
orderfees_shipping.php	install(), uninstall(), getContent(), addnewfeatures(), simulatorTablesExist(), installSimulatorTables/Configuration/Tab(), cleanSimulatorData(), refreshRuleHash(), 7 claves nuevas en \$configurations, const FEATURES_VERSION.
config/menus.xml	Entrada del menú del simulador (instalaciones nuevas).
config/sql/uninstall.sql	DROP de las 5 tablas nuevas.
translations/es.php	108 cadenas del simulador + 2 claves menu_ que faltaban: la pestaña "Advanced Shipping cost" salía en inglés porque getTranslations() la busca con el prefijo menu_ y solo existía con el prefijo orderfees_shipping_.

Nota sobre actualizaciones del vendor: todo lo nuevo son ficheros **nuevos**, que un update de motionSeed no pisa. Lo único que se perdería es el parche de orderfees_shipping.php (y con él addnewfeatures()), pero para entonces la tab y las tablas ya existen en BD, así que el simulador **seguiría funcionando**. El aviso de rule_hash te dirá que el módulo se ha actualizado.

8. Limitaciones conocidas

- Los pedidos pasados no se pueden reproducir con exactitud.** Hora, tipo de cambio, stock y precios son los de hoy. Si el importe no cuadra y no hay reglas modificadas, sospecha de una regla con franja horaria o con conversión de divisa.
- El simulador reproduce fielmente los bugs del vendor**, y eso es intencionado. En particular, la caché de `ShippingRule::value()` no incluye el `id_carrier` (`ShippingRule.php:1195`), así que una fórmula que use la variable `shipping` puede dar el mismo valor para varios transportistas. **Eso pasa igual en producción** (`getCarriersForOrder()` evalúa todos los transportistas en la misma petición). Corregirlo en el simulador lo haría *divergir* del motor real, que es justo lo que no queremos.
- La variable `shipping` de las fórmulas no es el porte nativo de PrestaShop:** es el acumulador de las reglas anteriores, y vale **0** en la primera regla evaluada (`ShippingRule.php:1266` + `override/classes/Cart.php:20`). Es un comportamiento del vendor, no del simulador, pero conviene saberlo al leer la traza.
- Transportistas de tipo módulo** (`is_module = 1`): se marcan con una etiqueta. Un módulo transportista mal escrito podría hacer una llamada externa (cotización a la API del transportista) al calcular el precio. Si te preocupa, filtra por un transportista concreto.
- Coste de las sondas:** hasta 13 `check()` extra por regla rechazada. Con muchas reglas activas se puede alcanzar el tope de `MAX_PROBES` (300); en ese caso el simulador **te avisa** de que pueden faltar causas en vez de callárselo.